

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Nguyễn Công Long - Nguyễn Văn Quý

**Tìm hiểu và xây dựng blockchain
định hướng cho doanh nghiệp**

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CỬ NHÂN TÀI NĂNG

Tp. Hồ Chí Minh, tháng 08/2026

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Nguyễn Công Long - 22120191
Nguyễn Văn Quý - 22120305

Tìm hiểu và xây dựng blockchain định hướng cho doanh nghiệp

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CỬ NHÂN TÀI NĂNG

GIẢNG VIÊN HƯỚNG DẪN
TS. Trần Trung Dũng

Tp. Hồ Chí Minh, tháng 08/2026

Lời cam đoan

Chúng tôi xin cam đoan khóa luận tốt nghiệp “*Tìm hiểu và xây dựng blockchain định hướng cho doanh nghiệp*” là công trình nghiên cứu của chính chúng tôi dưới sự hướng dẫn của TS. Trần Trung Dũng. Các nội dung, số liệu và kết quả trình bày trong khóa luận là trung thực. Toàn bộ mã nguồn của hệ thống do nhóm tự thiết kế và hiện thực; trong đó thuật toán đồng thuận Raft được **tự cài đặt hoàn toàn từ đầu**, không sử dụng thư viện đồng thuận có sẵn. Những phần tham khảo từ tài liệu, thư viện mã nguồn mở và công trình của người khác đều được trích dẫn và liệt kê đầy đủ trong mục Tài liệu tham khảo.

Chúng tôi xin chịu hoàn toàn trách nhiệm về lời cam đoan này.

TP. Hồ Chí Minh, tháng 08 năm 2026

Nhóm tác giả

Nguyễn Công Long

Nguyễn Văn Quý

Lời cảm ơn

Trước hết, chúng tôi xin gửi lời cảm ơn chân thành và sâu sắc nhất đến TS. Trần Trung Dũng — giảng viên hướng dẫn — đã tận tình định hướng, góp ý và đồng hành cùng nhóm trong suốt quá trình thực hiện khóa luận. Những trao đổi về kiến trúc hệ phân tán, thuật toán đồng thuận và phương pháp đánh giá hiệu năng là nền tảng quan trọng để chúng tôi hoàn thành công trình này.

Chúng tôi xin cảm ơn quý Thầy Cô Khoa Công nghệ Thông tin, Trường Đại học Khoa học Tự nhiên — Đại học Quốc gia TP. Hồ Chí Minh, đặc biệt là Chương trình Cử nhân Tài năng, đã trang bị cho chúng tôi kiến thức nền tảng vững chắc về khoa học máy tính, mạng máy tính, hệ điều hành và mật mã học.

Cuối cùng, chúng tôi xin cảm ơn gia đình và bạn bè đã luôn ủng hộ, động viên trong suốt thời gian học tập và nghiên cứu.

Mặc dù đã nỗ lực hết sức, khóa luận không tránh khỏi những thiếu sót. Chúng tôi rất mong nhận được những ý kiến đóng góp quý báu từ quý Thầy Cô và bạn đọc.

Nhóm tác giả

Table of Contents

Acknowledgement	iii
Proposal	iii
List of Figures	v
List of Tables	vi
Abstract	1
1 Giới thiệu	3
1.1 Bối cảnh và động lực	3
1.2 Blockchain công khai và blockchain có cấp phép	4
1.3 Phát biểu bài toán	5
1.4 Mục tiêu và phạm vi	6
1.5 Đóng góp của khóa luận	7
1.6 Phương pháp tiếp cận	8
1.7 Kết quả đạt được	8
1.8 Cấu trúc khóa luận	8
2 Các công trình liên quan	10
2.1 Sở cái phân tán và hàm băm mật mã	10
2.2 Bitcoin và mô hình UTXO	11
2.3 Ethereum, mô hình tài khoản và hợp đồng thông minh	11
2.4 Hyperledger Fabric và mô hình Execute–Order–Validate	12
2.5 Bài toán đồng thuận và các thuật toán tiêu biểu	13
2.6 WebAssembly cho hợp đồng thông minh	15
2.7 Mạng ngang hàng và lưu trữ	16

2.8	Đánh giá hiệu năng blockchain	17
2.9	Định vị của khóa luận	17
3	Phương pháp và Thiết kế hệ thống	18
3.1	Tổng quan kiến trúc	18
3.2	Mô hình dữ liệu	21
3.3	Core Service — Cổng vào và Tầng tính toán	23
3.4	Ordering Service — Đồng thuận Raft	27
3.5	Committing Peer — Kiểm tra và Ghi sổ	34
3.6	Blockchain Explorer — Giao diện	37
3.7	Cơ sở dữ liệu PostgreSQL	38
3.8	Phương pháp đo lường hiệu năng	39
3.9	Tổng kết công nghệ	40
	References	42

List of Figures

1	Kiến trúc tổng thể và các luồng giao tiếp chính của hệ thống. . .	19
2	Giao diện hàm (ABI) giữa host và contract WASM.	24
3	Sơ đồ chuyển trạng thái của một node Raft trong dự án.	28
4	Quy trình bầu lãnh đạo theo độ ưu tiên.	30
5	Kiến trúc đường ống ba tầng của Committing Peer.	35

List of Tables

1	So sánh blockchain công khai và blockchain có cấp phép	5
2	Đối chiếu vai trò giữa Hyperledger Fabric và hệ thống của khóa luận	13
3	So sánh các thuật toán đồng thuận	15
4	Ba tầng trách nhiệm và ranh giới của chúng	18
5	Ba hợp đồng mẫu	25
6	Các endpoint chính của Core Service	27
7	So sánh Raft chuẩn và Raft của dự án	28
8	Các hằng số thời gian của tầng đồng thuận	29
9	Các tối ưu tốc độ trên đường nóng	32
10	Các Protocol ID libp2p dùng trong hệ thống	33
11	Ba lớp lưu trữ của Committing Peer	36
12	Công nghệ của Blockchain Explorer	37
13	Ba schema của cơ sở dữ liệu	38
14	Ba lớp đo lường hiệu năng	39
15	Kết quả benchmark tham khảo (5000 req/s, 60s)	40
16	Ảnh xạ “vấn đề ↔ công nghệ”	41

Abstract

Khóa luận này trình bày quá trình **tìm hiểu, thiết kế và hiện thực một nền tảng blockchain có cấp phép (permissioned blockchain) định hướng cho doanh nghiệp**, được xây dựng hoàn toàn từ đầu (from-scratch) và lấy cảm hứng từ kiến trúc của Hyperledger Fabric. Khác với phần lớn các đề án sử dụng lại một nền tảng blockchain có sẵn, mục tiêu cốt lõi của chúng tôi là *làm chủ công nghệ lõi*: tự cài đặt từng thành phần để hiểu thấu đáo cách một hệ thống sổ cái phân tán hoạt động, từ tầng mạng ngang hàng, thuật toán đồng thuận, máy ảo hợp đồng thông minh, cho đến tầng lưu trữ bất biến và world state.

Hệ thống được tổ chức theo triết lý tách biệt ba trách nhiệm **Execute—Order—Validate** của Hyperledger Fabric, gồm bốn thành phần độc lập giao tiếp qua mạng ngang hàng libp2p: (1) **Core Service** đóng vai trò cổng vào và tầng tính toán, thực thi hợp đồng thông minh biên dịch sang WebAssembly trong môi trường cô lập **wazero** và điều phối quá trình xác nhận (endorsement); (2) **Ordering Service** — trái tim đồng thuận của hệ thống — *tự cài đặt thuật toán Raft từ đầu* với một biến thể bầu lãnh đạo theo độ ưu tiên (priority-based) thay cho cơ chế timeout ngẫu nhiên; (3) **Committing Peer** kiểm tra tính hợp lệ về mật mã của block và ghi vĩnh viễn vào sổ cái append-only kèm world state UTXO trên LevelDB; và (4) **Blockchain Explorer** là giao diện web React hiển thị dữ liệu theo thời gian thực qua Server-Sent Events. Một cơ sở dữ liệu PostgreSQL đóng vai trò bản sao (mirror) bất đồng bộ phục vụ tra cứu và đo lường, hoàn toàn nằm ngoài luồng đồng thuận.

Khóa luận đặc biệt đi sâu vào tầng đồng thuận: mô hình hóa các trạng thái của node, quy trình bầu lãnh đạo xác định, cơ chế nhân bản log và cắt block theo cơ chế lai (hybrid) giữa kích thước lô và thời gian, quá trình giao block (deliver) theo mô hình đăng-ký-phát-tỏa, và cơ chế đồng bộ (sync) bốn pha dựa trên đồng thuận (**commitIndex**, **commitHash**). Chúng tôi cũng trình bày một loạt tối ưu hiệu năng trên đường nóng (hot path) giúp hệ thống tiệm cận mục tiêu **5.000 giao dịch mỗi giây**, cùng phương pháp đo lường ba lớp (k6, submit, commit) để phân tích throughput, độ trễ end-to-end và hiện tượng backlog dưới tải bão hòa.

Từ khóa: blockchain có cấp phép, Hyperledger Fabric, đồng thuận Raft, hợp đồng thông minh WebAssembly, mạng ngang hàng libp2p, mô hình Execute-Order-Validate, sổ cái phân tán, throughput, độ trễ.

Chapter 1

Giới thiệu

1.1 Bối cảnh và động lực

Trong hơn một thập kỷ qua, kể từ khi Satoshi Nakamoto công bố Bitcoin vào năm 2008 [1], công nghệ chuỗi khối (blockchain) đã phát triển từ một hệ thống tiền điện tử đơn lẻ thành một họ công nghệ nền tảng cho nhiều lĩnh vực: tài chính phi tập trung, truy xuất nguồn gốc hàng hóa, quản lý tài sản số, định danh phi tập trung và hợp đồng thông minh. Bản chất của blockchain — một *sổ cái phân tán* (distributed ledger) bất biến, được nhiều bên không hoàn toàn tin tưởng nhau cùng duy trì — mang lại ba thuộc tính đặc biệt giá trị với doanh nghiệp và nhà nước: tính **minh bạch**, tính **bất biến** và khả năng **kiểm toán** mà không cần một bên trung gian duy nhất nắm giữ quyền lực tuyệt đối.

Sự trưởng thành của công nghệ này đã được phản ánh trong định hướng chính sách ở tầm quốc gia. Nghị quyết số 57-NQ/TW ngày 22/12/2024 của Bộ Chính trị về phát triển khoa học, công nghệ, đổi mới sáng tạo và chuyển đổi số quốc gia đã xác định blockchain là một trong các công nghệ chiến lược cần ưu tiên làm chủ [2]. Trước đó, Quyết định số 1236/QĐ-TTg ngày 30/6/2023 đã phê duyệt *Chiến lược quốc gia về ứng dụng và phát triển công nghệ blockchain đến năm 2030* [3], trong đó đặt ra mục tiêu xây dựng hạ tầng mạng blockchain quốc gia “Make in Vietnam” với năng lực xử lý cao, độ trễ thấp và khả năng mở rộng tới hàng trăm nút xác thực phân tán.

Một trong những yêu cầu kỹ thuật then chốt được nêu ra trong các đề án phát triển hạ tầng blockchain quốc gia là khả năng xử lý **tối thiểu 5.000 giao dịch mỗi giây** với độ trễ dưới một giây, đồng thời đảm bảo an toàn dữ liệu và khả năng mở rộng [3]. Đây không phải là những con số dễ đạt được: chúng đòi hỏi hiểu biết sâu sắc về kiến trúc hệ phân tán, thuật toán đồng thuận, tối ưu hiệu năng trên đường truyền mạng và tầng lưu trữ. Quan trọng hơn, để thực sự *làm chủ* công nghệ — chứ không chỉ vận hành lại một sản phẩm mã nguồn mở của nước ngoài — người kỹ sư cần hiểu được từng lớp của hệ thống hoạt động

ra sao, vì sao lại thiết kế như vậy, và đâu là các điểm đánh đổi (trade-off) cốt lõi.

Chính trong bối cảnh đó, khóa luận này đặt ra một mục tiêu học thuật rõ ràng: *tự xây dựng một nền tảng blockchain định hướng doanh nghiệp từ đầu, để hiểu thấu đáo cơ chế bên trong của một hệ thống sổ cái phân tán có cấp phép*. Chúng tôi không nhằm cạnh tranh với các nền tảng công nghiệp đã được kiểm chứng như Hyperledger Fabric [4], mà coi việc tái hiện các nguyên lý thiết kế của chúng — đặc biệt là việc **tự cài đặt thuật toán đồng thuận Raft** [5] — như một con đường để làm chủ tri thức nền tảng, một bước chuẩn bị cho việc tham gia phát triển hạ tầng blockchain quốc gia trong tương lai.

1.2 Blockchain công khai và blockchain có cấp phép

Không phải mọi blockchain đều giống nhau. Một sự phân loại quan trọng — và là nền tảng cho toàn bộ thiết kế của khóa luận — là sự phân biệt giữa blockchain **công khai** (public/permissionless) và blockchain **có cấp phép** (permissioned).

Blockchain công khai như Bitcoin [1] hay Ethereum [6] cho phép bất kỳ ai cũng có thể tham gia mạng, đọc dữ liệu, gửi giao dịch và tham gia vào quá trình đồng thuận một cách ẩn danh. Vì các bên tham gia không có danh tính rõ ràng và có thể có hành vi xấu, các hệ thống này phải dựa vào những cơ chế đồng thuận tốn kém như Proof-of-Work (bằng chứng công việc) — vốn đòi hỏi một lượng năng lượng tính toán khổng lồ — để chống lại tấn công Sybil và đảm bảo an toàn.

Ngược lại, blockchain có cấp phép — nhóm mà hệ thống trong khóa luận này thuộc về — chỉ cho phép các thành viên *được cấp phép* tham gia. Mỗi node có một danh tính mật mã rõ ràng, thường được biểu diễn qua một cặp khóa công khai/bí mật. Nhờ giả định về danh tính này, hệ thống có thể sử dụng các thuật toán đồng thuận nhẹ hơn nhiều (như Raft hoặc PBFT [7]), đạt throughput cao hơn và độ trễ thấp hơn đáng kể so với các blockchain công khai. Bảng 1 tóm tắt những khác biệt cốt lõi.

Table 1. So sánh blockchain công khai và blockchain có cấp phép

Tiêu chí	Công khai	Có cấp phép (dự án này)
Ví dụ điển hình	Bitcoin, Ethereum	Hyperledger Fabric, hệ thống này
Ai được tham gia	Bất kỳ ai	Chỉ thành viên được cấp phép
Định danh	Ẩn danh / bút danh	Danh tính mật mã rõ ràng
Cơ chế đồng thuận	Tổn kém (PoW, PoS)	Nhẹ (Raft, PBFT)
Throughput	Thấp (vài chục TPS)	Cao (hàng nghìn TPS)
Độ trễ	Cao (phút)	Thấp (giây / mili-giây)
Mô hình tin cậy	Không tin cậy nhau	Tin cậy một phần

Mô hình có cấp phép phù hợp tự nhiên với ngữ cảnh doanh nghiệp và khu vực công: các tổ chức tham gia (ngân hàng, cơ quan quản lý, nhà cung cấp trong chuỗi cung ứng) đều biết danh tính của nhau, chịu sự ràng buộc pháp lý, và cần một hệ thống vừa minh bạch vừa có hiệu năng đủ cao để phục vụ nghiệp vụ thực tế. Đây chính là phân khúc mà Hyperledger Fabric nhắm tới, và cũng là định hướng của khóa luận.

1.3 Phát biểu bài toán

Xây dựng một nền tảng blockchain doanh nghiệp hoàn chỉnh từ đầu đặt ra một loạt thách thức kỹ thuật đan xen nhau, mà nếu chỉ sử dụng lại một nền tảng có sẵn thì người học sẽ không bao giờ phải đối mặt một cách trực tiếp:

1. **Bài toán đồng thuận.** Khi nhiều node cùng nhận giao dịch gần như đồng thời, làm thế nào để tất cả thống nhất về một *thứ tự toàn phần* (total order) duy nhất của các giao dịch? Đây là bài toán đồng thuận kinh điển trong hệ phân tán, với kết quả bất khả thi nổi tiếng FLP [8] cho thấy không thể có thuật toán đồng thuận vừa an toàn vừa luôn kết thúc trong mạng bất đồng bộ có lỗi. Việc tự cài đặt một thuật toán đồng thuận đúng đắn (ở đây là Raft) đòi hỏi xử lý cẩn thận các tình huống bầu cử, mất kết nối, và phân ly mạng (split-brain).
2. **Bài toán thực thi hợp đồng thông minh an toàn.** Hợp đồng thông minh phải chạy một cách *xác định* (deterministic) — cùng đầu vào luôn cho cùng kết quả trên mọi node — và *cô lập* (sandboxed) để không thể phá hoại hệ thống chủ. Làm thế nào để hỗ trợ nhiều ngôn ngữ lập trình mà vẫn đảm bảo hai thuộc tính này?

3. **Bài toán xác thực và toàn vẹn.** Làm thế nào để xác thực “ai gửi giao dịch” mà không cần mật khẩu, và đảm bảo dữ liệu sổ cái không thể bị sửa đổi một cách âm thầm?
4. **Bài toán hiệu năng.** Làm thế nào để đạt throughput hàng nghìn giao dịch mỗi giây trong khi vẫn giữ độ trễ ở mức chấp nhận được? Đây là các nút nghẽn cổ chai (bottleneck) trên đường truyền, và làm thế nào để nhận diện cũng như loại bỏ chúng?
5. **Bài toán giao tiếp mạng.** Làm thế nào để các thành phần phân tán giao tiếp một cách an toàn, có định danh mật mã, và chịu được việc node tham gia/rời mạng động?

Mỗi thách thức trên tương ứng với một quyết định thiết kế cụ thể trong hệ thống. Khóa luận này đặt mục tiêu *trình bày và lý giải* từng quyết định đó, dựa trên mã nguồn thực tế đã được hiện thực và kiểm thử.

1.4 Mục tiêu và phạm vi

1.4.1 Mục tiêu

Mục tiêu tổng quát của khóa luận là **thiết kế, hiện thực và đánh giá một nền tảng blockchain có cấp phép định hướng doanh nghiệp, tự xây dựng từ đầu**. Mục tiêu này được cụ thể hóa thành các mục tiêu thành phần:

- Tự cài đặt thuật toán đồng thuận **Raft** từ đầu (không dùng thư viện), bao gồm bầu lãnh đạo, nhân bản log, cắt block và đồng bộ — với một biến thể bầu lãnh đạo theo độ ưu tiên.
- Xây dựng một máy ảo hợp đồng thông minh dựa trên **WebAssembly**, cho phép viết hợp đồng bằng ngôn ngữ bậc cao và thực thi an toàn, xác định.
- Hiện thực mô hình xác nhận giao dịch **Execute–Order–Validate** theo triết lý của Hyperledger Fabric, với chữ ký số Ed25519 và kiểm tra endorsement.
- Xây dựng tầng lưu trữ bất biến (append-only) cho chuỗi block và world state UTXO trên LevelDB với cập nhật nguyên tử.
- Xây dựng giao diện web (Explorer) hiển thị dữ liệu theo thời gian thực.
- Thiết lập phương pháp đo lường hiệu năng và đánh giá hệ thống dưới tải cao, hướng tới mục tiêu 5.000 TPS.

1.4.2 Phạm vi

Để đảm bảo chiều sâu, khóa luận tập trung vào **luồng xử lý cốt lõi** của một giao dịch từ lúc được gửi đến lúc được ghi vĩnh viễn. Cụ thể, báo cáo đi sâu vào bốn thành phần: Core Service, Ordering Service (trọng tâm), Committing Peer và Blockchain Explorer, cùng cơ sở dữ liệu PostgreSQL đóng vai trò mirror.

Một số thành phần phụ trợ phục vụ vận hành — như công cụ điều phối cụm (orchestrator) và giao diện giám sát cụm trong thư mục `orderingservice/` — nằm **ngoài phạm vi** trình bày, vì chúng là công cụ quản trị, không thuộc luồng xử lý giao dịch chính. Tương tự, khóa luận tập trung vào mô hình tin cậy của blockchain có cấp phép (chống lỗi dừng — crash fault tolerance), và chỉ *thảo luận* (chứ không hiện thực đầy đủ) khả năng chịu lỗi Byzantine.

1.5 Đóng góp của khóa luận

Những đóng góp chính của khóa luận bao gồm:

1. **Một hiện thực Raft hoàn chỉnh, tự viết**, với biến thể bầu lãnh đạo theo độ ưu tiên (priority-based leader election) thay cho cơ chế timeout ngẫu nhiên của Raft chuẩn. Biến thể này loại bỏ hiện tượng chia phiếu (split vote) và cho phép hội tụ xác định, đồng thời vẫn giữ nguyên tắc an toàn dựa trên quorum đa số trên tổng số thành viên để chống split-brain.
2. **Một kiến trúc tách tầng Execute–Order–Validate** được hiện thực bằng bốn tiến trình độc lập giao tiếp qua `libp2p`, minh họa cụ thể cách triết lý thiết kế của Hyperledger Fabric vận hành trong thực tế.
3. **Một máy ảo hợp đồng thông minh WebAssembly** với cơ chế cache module đã biên dịch và pool sandbox dựng sẵn, đạt khả năng thực thi song song hiệu năng cao.
4. **Một loạt tối ưu hiệu năng trên đường nóng** (được đánh số OPT-1 đến OPT-8), giúp tăng số giao dịch trên mỗi block từ khoảng 71 lên hơn 1000 dưới tải bền vững, đưa hệ thống tiệm cận mục tiêu 5.000 TPS.
5. **Một phương pháp đo lường hiệu năng ba lớp** (k6, submit, commit) cho phép phân biệt rõ ràng độ trễ chấp nhận của cổng vào với độ trễ end-to-end, và giải thích hiện tượng backlog dưới tải bão hòa thông qua định luật Little [9].

1.6 Phương pháp tiếp cận

Khóa luận áp dụng phương pháp *xây dựng để hiểu* (build-to-understand): thay vì chỉ phân tích lý thuyết, chúng tôi hiện thực toàn bộ hệ thống bằng ngôn ngữ Go cho ba dịch vụ backend và React cho giao diện, sau đó tiến hành kiểm thử chức năng và đo lường hiệu năng thực nghiệm. Mỗi quyết định thiết kế đều được đối chiếu với (a) nguyên lý lý thuyết tương ứng (bài báo Raft, kiến trúc Fabric, đặc tả WebAssembly...) và (b) kết quả đo lường thực tế. Cách tiếp cận này cho phép phát hiện và lý giải các điểm đánh đổi mà chỉ khi tự hiện thực mới bộc lộ ra, ví dụ như nghẽn cổ chai do tranh chấp khóa khi ghi log theo từng giao dịch, hay sự khác biệt giữa độ trễ HTTP và độ trễ ghi sổ thực sự.

1.7 Kết quả đạt được

Hệ thống đã được hiện thực hoàn chỉnh và chạy được luồng giao dịch end-to-end. Trong các phép đo tải tham khảo (gửi 5.000 yêu cầu mỗi giây trong 60 giây), hệ thống đạt:

- Tốc độ chấp nhận (submit) bền vững khoảng 4.691 giao dịch/giây, đỉnh khoảng 5.017 giao dịch/giây.
- Tốc độ ghi sổ (commit) bền vững khoảng 3.921 giao dịch/giây, đỉnh khoảng 6.000 giao dịch/giây.
- Trung bình khoảng 989 giao dịch trên mỗi block (gần đúng kích thước lô mục tiêu 1.000).

Các kết quả này, cùng với phân tích hiện tượng backlog khi tốc độ gửi vượt tốc độ ghi, được trình bày chi tiết ở chương đánh giá. Chúng cũng là cơ sở để đề xuất các hướng cải thiện trong các chương sau.

1.8 Cấu trúc khóa luận

Phần còn lại của khóa luận được tổ chức như sau:

- **Chương 2 — Các công trình liên quan:** trình bày nền tảng lý thuyết và các công trình tiền đề: Bitcoin và mô hình UTXO, Ethereum và hợp đồng thông minh, Hyperledger Fabric và mô hình Execute–Order–Validate, các

thuật toán đồng thuận (Paxos, Raft, PBFT), WebAssembly cho hợp đồng thông minh, và định vị của khóa luận so với các công trình này.

- **Chương 3 — Phương pháp và Thiết kế hệ thống:** trình bày chi tiết kiến trúc tổng thể, mô hình dữ liệu, và thiết kế của từng thành phần (Core Service, Ordering Service, Committing Peer, Explorer), cơ sở dữ liệu, cùng phương pháp đo lường hiệu năng.
- Các chương tiếp theo (sẽ được hoàn thiện) trình bày hiện thực chi tiết, kết quả thực nghiệm, đánh giá và các đề xuất cải thiện về tốc độ, lưu trữ và bảo mật.

Chapter 2

Các công trình liên quan

Chương này khảo sát nền tảng lý thuyết và các công trình tiền đề mà khóa luận dựa vào hoặc lấy cảm hứng. Chúng tôi đi từ những khái niệm nền tảng nhất (sổ cái phân tán, hàm băm, cây Merkle) đến các hệ thống blockchain tiêu biểu (Bitcoin, Ethereum, Hyperledger Fabric), rồi tới các thuật toán đồng thuận, công nghệ máy ảo hợp đồng thông minh và hạ tầng mạng ngang hàng. Cuối chương, chúng tôi định vị hệ thống của khóa luận trong bức tranh tổng thể đó.

2.1 Sổ cái phân tán và hàm băm mật mã

Cốt lõi của mọi blockchain là một **sổ cái phân tán**: một danh sách các bản ghi (giao dịch) được nhóm thành các *block* (khối), trong đó mỗi block chứa “dấu vân tay” mật mã của block liền trước. Dấu vân tay này được tính bằng một **hàm băm mật mã** (cryptographic hash function) — một hàm biến dữ liệu có độ dài tùy ý thành một chuỗi bit có độ dài cố định, với ba thuộc tính then chốt: kháng tiền ảnh (preimage resistance), kháng tiền ảnh thứ hai, và kháng va chạm (collision resistance). Một thay đổi dù chỉ một bit ở dữ liệu đầu vào cũng làm thay đổi hoàn toàn giá trị băm, và về mặt tính toán là không khả thi để đảo ngược.

Vì mỗi block tham chiếu tới hash của block trước, việc sửa đổi một block cũ sẽ làm thay đổi hash của nó, kéo theo phải tính lại toàn bộ các block sau — điều khiến dữ liệu trở nên gần như **bất biến**. Đây chính là cơ chế “chuỗi” (chain) trong từ “blockchain”. Hệ thống trong khóa luận sử dụng SHA-256 (cụ thể là double-SHA256, giống Bitcoin) [10,1] cho hash block.

2.1.1 Cây Merkle

Để tóm tắt toàn bộ các giao dịch trong một block thành một giá trị duy nhất, blockchain sử dụng **cây Merkle** [11] (Merkle tree, hay hash tree). Trong cấu trúc này, các giao dịch là lá; mỗi nút trong là hash của hai nút con ghép

lại; và nút gốc — *Merkle root* — tóm tắt toàn bộ cây. Cây Merkle mang lại hai lợi ích: (1) bất kỳ thay đổi nào ở một giao dịch đều làm thay đổi Merkle root, nên root đóng vai trò bằng chứng toàn vẹn; và (2) người ta có thể chứng minh một giao dịch thuộc về block mà chỉ cần cung cấp $O(\log n)$ hash trên đường từ lá đến gốc, thay vì tải toàn bộ block — cơ sở cho các “light client” (máy khách nhẹ). Hệ thống của khóa luận tính Merkle root từ danh sách `txid` của các giao dịch và đưa vào header của block.

2.2 Bitcoin và mô hình UTXO

Bitcoin [1] là blockchain đầu tiên giải quyết được bài toán “chi tiêu hai lần” (double-spending) trong môi trường không cần tin cậy, thông qua cơ chế đồng thuận Proof-of-Work và một cấu trúc dữ liệu giao dịch độc đáo: mô hình **UTXO** (Unspent Transaction Output — đầu ra giao dịch chưa tiêu).

Thay vì lưu “số dư tài khoản” như ngân hàng truyền thống, Bitcoin lưu một tập hợp các “tờ tiền” chưa tiêu. Mỗi giao dịch gồm:

- Các **đầu vào** (inputs, ký hiệu `Vin`): tham chiếu và “tiêu” các UTXO đã tồn tại từ trước.
- Các **đầu ra** (outputs, ký hiệu `Vout`): tạo ra các UTXO mới, gán cho người nhận.

Số dư của một địa chỉ chính là tổng giá trị các UTXO mà địa chỉ đó kiểm soát. Mô hình UTXO có ưu điểm về khả năng song song hóa kiểm tra giao dịch và tính riêng tư, nhưng kém trực quan hơn mô hình tài khoản (account model) khi biểu diễn trạng thái phức tạp. Hệ thống của khóa luận **hỗ trợ mô hình UTXO**: world state ở Committing Peer được tổ chức như một tập UTXO, và CLI client đóng vai trò một ví UTXO đầy đủ.

2.3 Ethereum, mô hình tài khoản và hợp đồng thông minh

Ethereum [6,12] mở rộng ý tưởng blockchain từ một hệ thống thanh toán thành một “máy tính thế giới” (world computer) có khả năng lập trình. Đóng

góp lớn nhất của Ethereum là **hợp đồng thông minh** (smart contract): các chương trình được lưu và thực thi trên blockchain, tự động chạy khi có giao dịch gọi tới.

Ethereum sử dụng **mô hình tài khoản** (account-based model) thay cho UTXO: trạng thái toàn cục là một ánh xạ từ địa chỉ tài khoản đến số dư và bộ nhớ hợp đồng. Hợp đồng được biên dịch sang bytecode của *Máy ảo Ethereum* (EVM) và thực thi theo mô hình **Order-Execute**: các giao dịch được sắp xếp trước (qua đồng thuận), sau đó *mọi* node thực thi lại tuần tự để cập nhật trạng thái. Mô hình này đơn giản về mặt khái niệm nhưng hạn chế về hiệu năng và khả năng song song: mọi node phải chạy lại mọi hợp đồng, và một hợp đồng độc hại (ví dụ vòng lặp vô hạn) có thể gây nguy hiểm — vấn đề mà Ethereum giải quyết bằng cơ chế “gas” tính phí cho từng bước tính toán.

Hệ thống của khóa luận hỗ trợ hợp đồng thông minh (qua WebAssembly thay vì EVM) nhưng áp dụng mô hình thực thi khác — Execute-Order-Validate của Fabric — sẽ được phân tích ở phần sau.

2.4 Hyperledger Fabric và mô hình Execute–Order–Validate

Hyperledger Fabric [4] là nền tảng blockchain có cấp phép tiêu biểu cho doanh nghiệp, và là **nguồn cảm hứng kiến trúc chính** của khóa luận. Đóng góp quan trọng nhất của Fabric — so với mô hình Order-Execute của Ethereum — là kiến trúc **Execute–Order–Validate**, tách quá trình xử lý một giao dịch thành ba pha độc lập:

1. **Execute (Thực thi & Xác nhận).** Giao dịch được gửi tới các *endorsing peer*; mỗi peer thực thi hợp đồng (*chaincode*) trong môi trường cô lập và *ký xác nhận* (endorse) kết quả, nhưng **chưa** ghi vào sổ cái. Vì các peer thực thi độc lập, bước này có thể được song song hóa.
2. **Order (Sắp xếp).** Các giao dịch đã có chữ ký xác nhận được gửi tới *ordering service*, nơi quyết định thứ tự toàn phần và gom chúng thành block. Đáng chú ý, ordering service **không cần hiểu nội dung** giao dịch — nó chỉ sắp xếp, nên rất nhẹ và dễ mở rộng.

3. **Validate (Kiểm tra & Ghi).** Các *committing peer* nhận block đã sắp xếp, kiểm tra chữ ký xác nhận có thỏa mãn chính sách endorsement không, kiểm tra xung đột phiên bản (đọc-ghi), rồi mới ghi vĩnh viễn.

Mô hình này mang lại ba lợi ích cốt lõi: (a) **khả năng song song** ở pha thực thi; (b) **tính module hóa**, cho phép thay thế cơ chế đồng thuận mà không ảnh hưởng tới logic nghiệp vụ; và (c) **bảo mật theo lớp**, vì mỗi tầng chỉ thấy thông tin nó cần. Fabric thật sử dụng thư viện etcd/raft cho ordering service.

Hệ thống của khóa luận **tái hiện trung thành** triết lý này: Core Service đóng vai trò endorsing peer/gateway (pha Execute), Ordering Service đóng vai trò ordering service (pha Order), và Committing Peer đóng vai trò committing peer (pha Validate). Điểm khác biệt học thuật quan trọng là chúng tôi **tự cài đặt thuật toán Raft từ đầu** thay vì dùng thư viện, và đơn giản hóa một số khía cạnh (ví dụ chính sách endorsement) để tập trung vào việc làm chủ cơ chế lỗi. Bảng 2 đối chiếu các vai trò.

Table 2. Đối chiếu vai trò giữa Hyperledger Fabric và hệ thống của khóa luận

Vai trò trong Fabric	Thành phần trong dự án	Pha
Endorsing Peer / Gateway	Core Service	Execute
Ordering Service (etcd/raft)	Ordering Service (Raft tự viết)	Order
Committing Peer	Committing Peer	Validate
Chaincode (Go/Java/Node)	Hợp đồng WASM (TinyGo)	Execute

2.5 Bài toán đồng thuận và các thuật toán tiêu biểu

Đồng thuận (consensus) là bài toán trung tâm của hệ phân tán: làm thế nào để nhiều tiến trình, một số có thể bị lỗi, cùng thống nhất về một giá trị (hoặc một chuỗi giá trị) duy nhất. Kết quả bất khả thi FLP [8] chứng minh rằng trong một hệ bất đồng bộ hoàn toàn, không tồn tại thuật toán đồng thuận *tất định* vừa luôn an toàn vừa luôn kết thúc khi có dù chỉ một tiến trình lỗi. Các thuật toán thực tế “né” kết quả này bằng cách dựa vào giả định thời gian (timeout) để đảm bảo tính kết thúc (liveness) trong khi luôn giữ tính an toàn (safety).

2.5.1 Paxos

Paxos [13] của Leslie Lamport là thuật toán đồng thuận tất định đầu tiên được chứng minh đúng đắn, và là nền tảng lý thuyết cho nhiều hệ thống. Tuy nhiên, Paxos nổi tiếng là *khó hiểu* và khó hiện thực đúng, đặc biệt khi mở rộng thành Multi-Paxos để đồng thuận một chuỗi giá trị.

2.5.2 Raft

Raft [5] được Ongaro và Ousterhout thiết kế với mục tiêu tường minh là *dễ hiểu* (understandability) mà vẫn tương đương Paxos về năng lực. Raft phân rã bài toán đồng thuận thành ba bài toán con tách biệt:

- **Bầu lãnh đạo (Leader Election):** tại mỗi thời điểm, chọn một node làm *Leader*; các node khác là *Follower*. Thời gian được chia thành các *term* (nhiệm kỳ), mỗi term có tối đa một Leader. Raft chuẩn dùng **timeout ngẫu nhiên**: Follower nào hết giờ chờ trước sẽ trở thành *Candidate* và xin phiếu bầu.
- **Nhân bản log (Log Replication):** chỉ Leader nhận lệnh từ client, ghi vào log của mình, rồi sao chép cho Follower. Khi một entry được sao chép tới **đa số** (majority) node, nó được coi là đã *commit*.
- **An toàn (Safety):** một loạt ràng buộc (chỉ bầu cho candidate có log đủ mới, chỉ commit entry của term hiện tại...) đảm bảo không có hai Leader hợp lệ cùng term và log không bị rẽ nhánh.

Khái niệm **quorum đa số** ($\lfloor N/2 \rfloor + 1$ trên tổng N node) là then chốt: vì hai tập đa số bất kỳ luôn giao nhau ở ít nhất một node, không thể có hai quyết định mâu thuẫn được commit. Nhờ vậy Raft chịu được tối đa $\lfloor (N - 1)/2 \rfloor$ node lỗi dừng. Raft là loại đồng thuận **chịu lỗi dừng** (Crash Fault Tolerant — CFT), giả định các node không có hành vi độc hại, chỉ có thể dừng hoặc chậm.

Khóa luận **tự cài đặt Raft** nhưng thay cơ chế bầu cử bằng timeout ngẫu nhiên bằng một biến thể **bầu lãnh đạo theo độ ưu tiên** (xem Chương 3), nhằm loại bỏ hiện tượng chia phiếu và đạt hội tụ xác định.

2.5.3 PBFT và đồng thuận chịu lỗi Byzantine

Trong môi trường mà các node có thể có hành vi *tùy ý* (kể cả độc hại) — bài toán Tướng Byzantine [14] — người ta cần các thuật toán **chịu lỗi Byzantine**

(Byzantine Fault Tolerant — BFT). PBFT [7] của Castro và Liskov là thuật toán BFT thực tế đầu tiên, chịu được tối đa $\lfloor (N - 1)/3 \rfloor$ node độc hại với chi phí truyền thông $O(N^2)$ thông điệp mỗi vòng. So với CFT, BFT an toàn hơn trong môi trường ít tin cậy nhưng đắt hơn nhiều về truyền thông và phức tạp hơn khi hiện thực.

Cachin và Vukolić [15] đã khảo sát một cách hệ thống các giao thức đồng thuận trong bối cảnh blockchain, chỉ ra rằng việc lựa chọn CFT hay BFT phụ thuộc vào mô hình tin cậy của ứng dụng. Với một blockchain có cấp phép trong nội bộ một liên minh doanh nghiệp đã định danh và chịu ràng buộc pháp lý, mô hình CFT (như Raft) thường là đủ và được ưa chuộng vì hiệu năng cao — đây cũng là lựa chọn của Fabric (với Raft) và của khóa luận này.

Table 3. So sánh các thuật toán đồng thuận

Thuật toán	Mô hình lỗi	Số node lỗi chịu được	Ghi chú
Paxos	Crash	$\lfloor (N - 1)/2 \rfloor$	Khó hiểu, nền tảng lý thuyết
Raft	Crash	$\lfloor (N - 1)/2 \rfloor$	Dễ hiểu; dùng trong dự án
PBFT	Byzantine	$\lfloor (N - 1)/3 \rfloor$	Chi phí $O(N^2)$
PoW (Bitcoin)	Byzantine	< 50% sức tính	Tốn năng lượng

2.6 WebAssembly cho hợp đồng thông minh

Việc lựa chọn công nghệ thực thi hợp đồng thông minh ảnh hưởng trực tiếp tới ba yêu cầu: tính xác định, tính cô lập và khả năng đa ngôn ngữ. **WebAssembly** (WASM) [16,17] là một định dạng mã máy ảo dạng stack, ban đầu được thiết kế cho trình duyệt web nhưng nhanh chóng trở thành lựa chọn phổ biến cho các môi trường thực thi phía máy chủ và blockchain. WASM thỏa cả ba yêu cầu: nó chạy trong một bộ nhớ tuyến tính cô lập (sandbox), có ngữ nghĩa thực thi xác định, và có thể được biên dịch từ nhiều ngôn ngữ bậc cao (Rust, Go, C/C++, AssemblyScript...).

Nhiều nền tảng blockchain hiện đại đã chuyển sang WASM: Polkadot dùng WASM cho runtime, CosmWasm đưa hợp đồng WASM vào hệ sinh thái Cosmos,

và Ethereum từng nghiên cứu eWASM như một thay thế cho EVM. Để giao tiếp với hệ điều hành chủ, WASM phía máy chủ thường dùng **WASI** (WebAssembly System Interface) [18] — một tập giao diện hệ thống chuẩn hóa.

Hệ thống của khóa luận biên dịch hợp đồng viết bằng Go sang WASM bằng **TinyGo**, và thực thi bằng **wazero** — một runtime WASM thuần Go không phụ thuộc thư viện C, giúp đơn giản hóa việc build và triển khai. Giữa host và contract có một giao diện hàm (ABI) quy ước để cấp phát bộ nhớ, truyền payload và ghi world state, sẽ được mô tả chi tiết ở Chương 3.

2.7 Mạng ngang hàng và lưu trữ

2.7.1 libp2p

Các thành phần của một blockchain phân tán cần giao tiếp an toàn và linh hoạt. Thay vì dùng HTTP/gRPC thông thường, hệ thống của khóa luận dùng **libp2p** [19] — ngăn xếp mạng module hóa được tách ra từ dự án IPFS [20] và cũng là nền mạng của Ethereum 2.0. libp2p cung cấp: (a) **định danh mật mã** (PeerID suy ra từ khóa công khai); (b) **địa chỉ tự mô tả** (multiaddr) như `/ip4/127.0.0.1/tcp/6000/p2p/12D3Koo...`; (c) **ghép kênh** (stream multiplexing) cho phép nhiều “làn” dữ liệu song song trên một kết nối; và (d) **thương lượng giao thức** qua các Protocol ID. Những đặc tính này đặc biệt phù hợp với giao tiếp consensus, nơi cần gửi nhiều loại thông điệp (heartbeat, đề xuất block, ACK...) một cách hiệu quả.

2.7.2 LevelDB và lưu trữ nhúng

Để lưu world state cần một cơ sở dữ liệu vừa nhanh vừa nhẹ. **LevelDB** [21] là một thư viện lưu trữ key-value nhúng, chạy ngay trong tiến trình (không cần server riêng), dựa trên cấu trúc LSM-tree (log-structured merge-tree) cho phép ghi tuần tự nhanh. Quan trọng nhất với blockchain, LevelDB hỗ trợ **ghi theo lô nguyên tử** (atomic batch): nhiều thao tác ghi/xóa hoặc được áp dụng toàn bộ, hoặc không gì cả — thuộc tính then chốt để cập nhật world state UTXO một cách nhất quán theo từng block.

2.8 Đánh giá hiệu năng blockchain

Việc đánh giá hiệu năng một blockchain có cấp phép đòi hỏi phương pháp luận cẩn thận. BLOCKBENCH [22] là một trong những khung đánh giá đầu tiên cho private blockchain, đề xuất đo throughput và độ trễ ở nhiều lớp khác nhau của hệ thống. Một bài học quan trọng là cần phân biệt rõ *nơi* đo: độ trễ mà client cảm nhận (HTTP) khác với độ trễ ghi sổ thực sự (end-to-end).

Khi tốc độ gửi giao dịch vượt quá tốc độ xử lý bền vững của hệ thống, hàng đợi giao dịch tích lũy thành **backlog** và độ trễ tăng vọt — hiện tượng có thể giải thích bằng **định luật Little** [9]: số lượng công việc trung bình trong hệ thống bằng tốc độ đến nhân với thời gian lưu trú trung bình ($L = \lambda W$). Khóa luận áp dụng cách đo ba lớp (k6 đo HTTP, submit đo lúc chấp nhận, commit đo lúc ghi sổ) và dùng định luật Little để lý giải hành vi backlog ở điểm bão hòa.

2.9 Định vị của khóa luận

Từ khảo sát trên, có thể định vị hệ thống của khóa luận như sau:

- **Về kiến trúc:** tái hiện mô hình Execute–Order–Validate của Hyperledger Fabric [4], nhưng đơn giản hóa để tập trung vào việc làm chủ cơ chế lỗi.
- **Về đồng thuận:** tự cài đặt Raft [5] từ đầu (so với Fabric dùng thư viện etcd/raft), với biến thể bầu lãnh đạo theo độ ưu tiên — một đóng góp mang tính học thuật, đánh đổi tính tổng quát lấy tính đơn giản và hội tụ xác định.
- **Về hợp đồng thông minh:** dùng WebAssembly [16] (giống Polkadot/CosmWasm) thay vì EVM, với runtime thuần Go.
- **Về mô hình giao dịch:** hỗ trợ cả UTXO (giống Bitcoin [1]) lẫn hợp đồng thông minh ghi world state (giống Ethereum [6]).

Khác biệt cốt lõi so với việc sử dụng lại một nền tảng có sẵn nằm ở *mục tiêu*: khóa luận không hướng tới một sản phẩm thương mại sẵn sàng vận hành, mà hướng tới việc **làm chủ tri thức nền tảng** thông qua việc tự hiện thực từng lớp. Chính vì vậy, báo cáo dành phần lớn dung lượng cho việc lý giải các quyết định thiết kế và các điểm đánh đổi, đặc biệt ở tầng đồng thuận — nội dung được trình bày chi tiết trong Chương 3.

Chapter 3

Phương pháp và Thiết kế hệ thống

Chương này trình bày chi tiết phương pháp luận thiết kế và toàn bộ kiến trúc của hệ thống. Chúng tôi bắt đầu từ cái nhìn tổng thể về kiến trúc bốn thành phần và mô hình Execute–Order–Validate, sau đó đặc tả mô hình dữ liệu chung, rồi đi sâu vào thiết kế của từng thành phần theo đúng hành trình của một giao dịch: Core Service (cổng vào và thực thi), Ordering Service (đồng thuận Raft — trọng tâm), Committing Peer (kiểm tra và ghi sổ), và Blockchain Explorer (giao diện). Cuối chương trình bày cơ sở dữ liệu PostgreSQL và phương pháp đo lường hiệu năng.

3.1 Tổng quan kiến trúc

3.1.1 Triết lý thiết kế: tách ba trách nhiệm

Hệ thống được tổ chức theo triết lý **Execute–Order–Validate** của Hyperledger Fabric (đã giới thiệu ở Mục 2.4), trong đó ba trách nhiệm cốt lõi của việc xử lý một giao dịch được tách thành ba tiến trình độc lập:

Table 4. Ba tầng trách nhiệm và ranh giới của chúng

Tầng	Thành phần	Làm gì	Không làm gì
Execute	Core Service	Chạy hợp đồng, xin endorsement	Không quyết định thứ tự
Order	Ordering Service (Raft)	Quyết định thứ tự, gom block	Không chạy hợp đồng, không xét nội dung
Validate	Committing Peer	Kiểm tra & ghi vĩnh viễn	Không sắp xếp lại thứ tự

Việc tách tiến trình mang lại bốn lợi ích thực tiễn: (1) **chịu lỗi** — Ordering Service gồm nhiều node, chết một node vẫn chạy nhờ Raft; (2) **mở rộng độc lập** — cần thêm sức tính hợp đồng thì thêm Core Service, cần ghi nhanh hơn thì thêm Committing Peer; (3) **bảo mật theo lớp** — mỗi tầng chỉ thấy thông tin

nó cần, ví dụ Ordering Service không cần biết nội dung giao dịch; và (4) **phản ánh đúng thực tế doanh nghiệp**, giống mô hình mà ngành tài chính và chuỗi cung ứng đang dùng.

3.1.2 Sơ đồ tổng thể

Hệ thống gồm bốn thành phần chạy độc lập, giao tiếp với nhau qua mạng ngang hàng libp2p, cùng một cơ sở dữ liệu PostgreSQL đóng vai trò bản sao tra cứu. Hình 1 mô tả kiến trúc và các luồng giao tiếp chính.

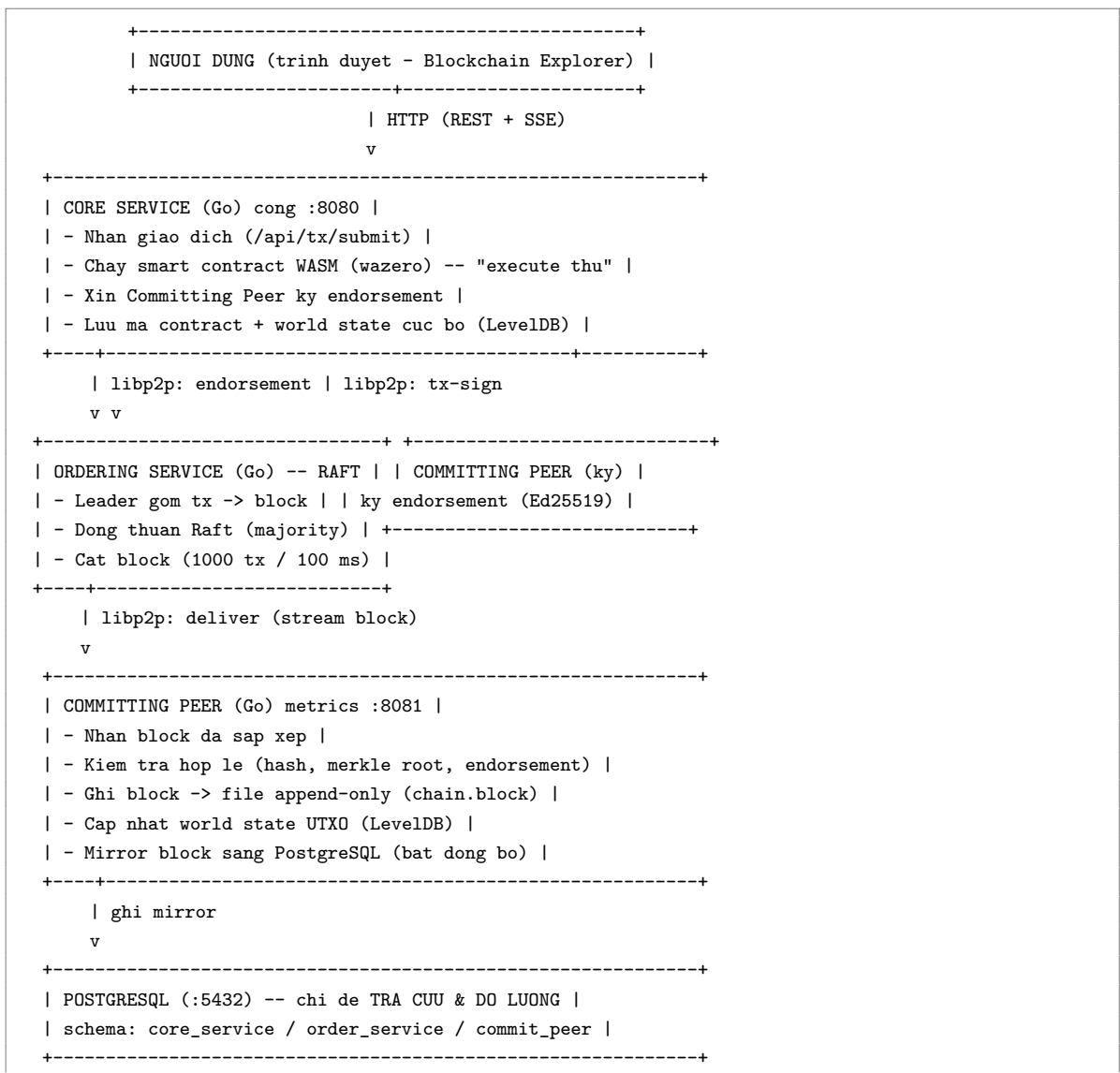


Fig. 1. Kiến trúc tổng thể và các luồng giao tiếp chính của hệ thống.

Một điểm thiết kế quan trọng: PostgreSQL **không** nằm trong luồng đồng thuận. Nó chỉ là bản sao (mirror) để Explorer tra cứu và để đo throughput/latency. Sổ cái “thật” nằm ở file `chain.block` và LevelDB của Committing Peer. Nếu PostgreSQL chết, blockchain vẫn chạy bình thường — một quyết định kiến trúc giúp tốc độ ghi sổ không bị phụ thuộc vào tốc độ của một cơ sở dữ liệu ngoài.

3.1.3 Hành trình của một giao dịch

Để hiểu cách các thành phần phối hợp, ta theo dõi hành trình end-to-end của một giao dịch gọi hợp đồng `demo_inventory` để đăng ký 100 sản phẩm:

1. **Tạo & gửi:** Người dùng điền form trên Explorer; frontend mã hóa dữ liệu thành payload nhị phân (hex) và gửi `POST /api/tx/submit` tới Core Service. Giao dịch gồm `txid`, `contract_name`, `function_name`, `payload`.
2. **Execute thử:** Core Service nạp WASM của hợp đồng, chạy hàm kiểm tra với payload. Thành công thì tiếp tục; thất bại trả lỗi ngay.
3. **Xin endorsement:** Core mở stream `tx-sign` tới Committing Peer; peer ký Ed25519 và trả lại giao dịch kèm chữ ký xác nhận.
4. **Gửi đi sắp xếp:** Core hỏi Discovery ai là Leader của cluster Raft, rồi gửi giao dịch (đã endorse) qua stream `endorsement` tới Leader; đồng thời ghi thời điểm gửi để đo latency.
5. **Đồng thuận & cắt block:** Leader gom giao dịch vào TxPool; khi đủ 1000 giao dịch *hoặc* sau 100ms, cắt một block, tính hash + Merkle root, gửi đề xuất cho Follower. Khi đa số Follower ACK, Leader commit block.
6. **Giao block:** Block vừa commit được phát tỏa (fan-out) qua stream `deliver` tới mọi Committing Peer.
7. **Kiểm tra & ghi:** Committing Peer kiểm tra hash, Merkle root, endorsement; nếu hợp lệ thì ghi block vào `chain.block` và cập nhật world state UTXO trong LevelDB (một thao tác ghi nguyên tử).
8. **Mirror & hiển thị:** Committing Peer ghi bản sao sang PostgreSQL bất đồng bộ; Core đọc lại và đẩy cập nhật về trình duyệt qua SSE để Explorer hiển thị gần như tức thì.

3.2 Mô hình dữ liệu

Trước khi đi vào từng thành phần, cần thống nhất các cấu trúc dữ liệu được dùng xuyên suốt hệ thống.

3.2.1 Giao dịch (Transaction)

Hệ thống hỗ trợ **đồng thời hai mô hình** giao dịch: mô hình hợp đồng thông minh (gọi hàm, ghi world state) và mô hình UTXO (Vin/Vout). Cấu trúc Transaction (trong `coreservice/internal/core/model.go`) phản ánh cả hai:

```
1 type EndorsementEntry struct {
2     PublicKey string // khoa công khai của peer xác nhận
3     Signature string // chữ ký
4 }
5
6 type Transaction struct {
7     Txid string
8     ContractName string
9     FunctionName string
10    Payload []byte // nhị phân, JSON mã hóa hex
11    Endorsements []EndorsementEntry // mảng chữ ký xác nhận
12    Signature string // (legacy) phân chiếu endorsement cuối
13    SenderPubKey string // (legacy) khoa của endorser cuối
14    Vin []VIN // họ tro UTXO: đầu vào
15    Vout []VOUT // họ tro UTXO: đầu ra
16    Version uint32
17    LockTime uint32
18 }
```

Listing 3.1. Cấu trúc Transaction và Endorsement

Hai trường `Signature` và `SenderPubKey` là kiểu cũ (một chữ ký); hệ thống mới ưu tiên mảng `Endorsements[]` để hỗ trợ *nhiều* chữ ký xác nhận. Khi serialize JSON, `Payload` được mã hóa hex thông qua các hàm `MarshalJSON/UnmarshalJSON` tùy chỉnh, nhằm biểu diễn dữ liệu nhị phân an toàn trong JSON.

3.2.2 Block

Một block tóm tắt một tập giao dịch đã được sắp xếp, với một header kiểu Bitcoin:

- `Timestamp`: thời điểm tạo block.
- `PrevHash`: hash của block trước (nối chuỗi).

- **MerkleRoot**: gốc cây Merkle tính từ danh sách **txid**.
- **Nonce**: giá trị phụ trong header.
- **Hash**: double-SHA256 của header — “dấu vân tay” của block.
- **Transactions**: danh sách giao dịch trong block.

Block hash được tính bằng **double-SHA256** của header (giống Bitcoin), và phải được tính *y* *hết* ở cả ba thành phần sinh/kiểm tra block (Ordering Service khi tạo, Committing Peer khi kiểm tra) để đảm bảo nhất quán.

3.2.3 World State và mô hình UTXO

World state là ảnh chụp trạng thái hiện tại của hệ thống, tính được từ toàn bộ lịch sử block. Trong khi chuỗi block là lịch sử bất biến (chỉ thêm), world state là bản đồ key→value có thể ghi đè. Với mô hình UTXO, world state là một *tập các UTXO chưa tiêu*, với lược đồ key:

$$\text{utxo}:\langle \text{txid} \rangle:\langle \text{vout_index} \rangle \longrightarrow \text{JSON của VOUT}$$

Khi áp dụng một block, mỗi VIN sẽ *xóa* một UTXO (đánh dấu đã tiêu), mỗi VOUT sẽ *tạo* một UTXO mới — toàn bộ trong một thao tác ghi nguyên tử (xem Mục 3.5.4).

3.2.4 Log entry của Raft

Ở tầng đồng thuận, mỗi đơn vị nhân bản là một **LogEntry** bao bọc một block:

```

1 type LogEntry struct {
2     Index int64 // vị trí trong log
3     PrevLogIndex int64 // index của entry liên trước (kiểm tra liên tục)
4     Term int64 // term khi tạo
5     Type string // "BLOCK_PROPOSING"
6     Block Block // block thực sự
7 }
```

Listing 3.2. Cấu trúc LogEntry trong Raft

Trường **PrevLogIndex** cho phép Follower kiểm tra log có **liên tục** hay không (không bị nhảy cóc) trước khi chấp nhận entry mới — đây là cốt lõi đảm bảo an toàn của Raft.

3.3 Core Service — Cổng vào và Tầng tính toán

3.3.1 Vai trò

Core Service (`coreservice/`, viết bằng Go, cổng HTTP :8080) là cổng vào và đồng thời là tầng tính toán của hệ thống, tương ứng vai trò Endorsing Peer/Gateway trong Fabric. Mọi giao dịch của người dùng đều đi qua đây đầu tiên. Ba nhiệm vụ chính: (1) thực thi hợp đồng thông minh WASM trong sandbox cô lập để “chạy thử”; (2) điều phối endorsement và gửi giao dịch tới Leader của Ordering Service; (3) phục vụ REST API, SSE realtime và tổng hợp số liệu benchmark.

Core Service gần như *không trạng thái* (stateless) đối với sổ cái: trạng thái chốt cuối nằm ở Committing Peer; Core chỉ giữ bản cache cục bộ (mã contract, world state demo) và đọc lại từ PostgreSQL khi cần hiển thị.

3.3.2 Luồng khởi động

Khi chạy (`cmd/node/main.go`), Core Service thực hiện tuần tự: (1) dựng libp2p host lắng nghe `/ip4/0.0.0.0/tcp/0`; (2) kết nối PostgreSQL với thử lại 10 lần — nếu thất bại vẫn chạy ở chế độ hạn chế; (3) mở hai instance LevelDB (`contract_db` và `ledger_db`), thử lại 3 lần và xóa file LOCK nếu kẹt; (4) tạo `WasmEngine` với pool module; (5) nhập địa chỉ Orderer & Committing Peer rồi làm âm kết nối; (6) đăng ký route HTTP và chạy server; (7) chờ tín hiệu dừng để tắt mượt.

3.3.3 Máy ảo hợp đồng thông minh WASM

Giao diện hàm (ABI) giữa host và contract Contract WASM và Core Service “nói chuyện” qua một số hàm quy ước. Contract xuất ra hai loại hàm cho host gọi: `allocate(size) -> ptr` để cấp phát bộ nhớ trong vùng tuyến tính của contract, và `verify_tx(ptr, size)` (hoặc `execute(ptr, size)`) là hàm chính trả về 1 (thành công) hoặc 0 (thất bại). Ngược lại, host cung cấp cho contract hàm `PutState(keyPtr, keySize, valPtr, valSize)` để ghi một cặp `key→value` vào world state. Hình 2 minh họa luồng gọi.

```

HOST (Core Service) CONTRACT (WASM)
| allocate(len(payload)) -----> cap phat, tra ptr
| ghi payload vào ptr
| verify_tx(ptr, len) -----> giai ma JSON, xu ly
| <----- PutState(key,val) (neu can ghi)
| nhan status (0/1) <----- return 1

```

Fig. 2. Giao diện hàm (ABI) giữa host và contract WASM.

Tối ưu hiệu năng: cache và pool Chạy WASM “từ đầu” mỗi giao dịch rất chậm vì phải biên dịch và khởi tạo module. `WasmEngine` (`internal/vm/engine.go`) dùng hai kỹ thuật:

1. **Cache module đã biên dịch:** hàm `getOrCompile()` biên dịch WASM một lần rồi giữ lại bản đã compile.
2. **Pool module dựng sẵn:** với mỗi contract, dựng sẵn N sandbox (mặc định 16, tối đa 32, biến `WASM_POOL_SIZE`) trong một channel có đệm. Mỗi giao dịch lấy một sandbox rảnh ra dùng rồi trả lại — tránh chi phí khởi tạo lặp lại và cho phép xử lý song song nhiều giao dịch.

Đây là mẫu thiết kế *object pool* kinh điển để giảm độ trễ khởi tạo.

Biên dịch contract Contract viết bằng Go được biên dịch sang WASM bằng TinyGo:

```

tinygo build -o my_contract.wasm -target wasi -no-debug -scheduler=none ./

```

Listing 3.3. Lệnh biên dịch contract

Trong đó `-target wasi` nhắm runtime phía máy chủ, và `-scheduler=none` bỏ bộ lập lịch goroutine cho nhẹ và xác định hơn.

Hợp đồng mẫu Dự án cung cấp ba hợp đồng mẫu phục vụ trình diễn và đo lường:

Table 5. Ba hợp đồng mẫu

Contract	Đầu vào (JSON)	Hành vi
example_asset	{id, color, action}	action=="create" → PutState("Asset_{id}")
demo_inventory	{op, sku, qty}	op=="register" và qty>=0 → PutState("Inv_{sku}")
bench_ping	{v}	Chỉ kiểm tra định dạng, không ghi state — đo throughput thuần

Hợp đồng `bench_ping` cố ý không ghi gì để đo “trần” tốc độ pipeline mà không bị nghẽn ở tầng lưu trữ.

3.3.4 Mật mã và quá trình endorsement

Hệ thống dùng chữ ký số **Ed25519** [23,24] — hệ chữ ký trên đường cong Edwards xoắn, nhanh, an toàn và xác định, với khóa 32 byte và chữ ký 64 byte. Hàm `SignTransaction` ghép `txID` + `contractName` + `payload` thành thông điệp rồi ký. Luồng xử lý `POST /api/tx/submit` như sau:

```

1 1. Giai ma JSON Transaction
2 2. engine.Execute() -> chạy contract WASM (validate)
   | thất bại -> trả lời ngay
   v thành công
3 3. signTxViaCommitPeer() -> mô stream tx-sign tới Committing Peer
   | Committing Peer ký Ed25519, thêm 1 EndorsementEntry
   v
4 4. sendEndorsementAsync() -> gửi giao dịch (đã endorse) tới Leader orderer
5 5. SubmitRecorder ghi tx_submit_times (đo latency)
6 6. Trả JSON: preview chu ký + số endorsement

```

Listing 3.4. Luồng xử lý submit giao dịch

Việc tách “execute thử” rồi mới ký chính là hiện thực mô hình `Execute–Order–Validate`: Core chạy thử để đảm bảo hợp lệ trước khi tốn công sắp xếp; `Committing Peer` ký xác nhận tạo bằng chứng mật mã; và khi block tới `Committing Peer` để ghi, nó *kiểm tra lại* các endorsement so với danh sách khóa tin cậy. Nhờ vậy, một giao dịch gian lận (không có endorsement hợp lệ) sẽ bị loại ở bước ghi, dù có lọt qua tầng sắp xếp. Bước 4 có thể chạy **bất đồng bộ** (`CORE_ASYNC_ENDORSE=1`) để trả lời người dùng nhanh hơn.

3.3.5 Mạng libp2p và Discovery

Mọi giao tiếp của Core với orderer và committing peer đều qua **stream** libp2p, phân loại bằng Protocol ID (xem Bảng 10). Hai cơ chế quan trọng cho hiệu năng và độ tin cậy:

Connection pool ấm: mở kết nối mới mỗi lần ký rất tốn (bắt tay TCP + libp2p). Core giữ sẵn một *kết nối ấm* tới Committing Peer (`CORE_SIGN_POOL`, mặc định bật); mỗi lần ký chỉ mở một *stream* mới trên kết nối sẵn có, rẻ hơn nhiều. Có cơ chế thử lại 2 lần và tự kết nối lại nếu hỏng.

Discovery — tìm Leader: Ordering Service có nhiều node và Leader có thể đổi. Discovery Client khởi tạo với danh sách bootstrap (vài multiaddr orderer), gọi `Refresh()` để lấy *membership view* (danh sách thành viên + ai là Leader) và cache lại (TTL ~8 giây). Khi phát hiện Leader cũ lỗi, `Invalidate()` xóa cache nóng và tạm dùng bản tốt cuối (`lastGood`) trong cửa sổ failover ~2 phút. Cơ chế cache + failover này giúp Core chịu được việc Leader đổi mà không cần khởi động lại.

3.3.6 Lưu trữ

Core dùng hai loại lưu trữ. **LevelDB** (nhúng cục bộ) mở hai instance: **ContractDB** lưu bytecode WASM và schema UI của contract, **LedgerDB** lưu world state demo. **PostgreSQL** là bản sao chỉ-đọc của sổ cái Committing Peer, phục vụ hiển thị Explorer, lưu danh mục contract và đo benchmark.

Một chi tiết tối ưu đáng chú ý là **SubmitRecorder** (`storage/submit_recorder.go`): để đo E2E latency cần ghi thời điểm submit của từng giao dịch, nhưng gọi DB cho *mỗi* giao dịch ở 5000 TPS sẽ làm sập DB. Giải pháp là **batch INSERT qua channel**: mỗi giao dịch chấp nhận đẩy (`txid`, `submitted_at`) vào một channel đệm lớn (~65k); một goroutine nền gom nhiều bản ghi rồi INSERT hàng loạt hoặc xả định kỳ. Đây là mẫu *batching* kinh điển để giảm số lần gọi I/O.

3.3.7 REST API và realtime

Core mở HTTP server tại :8080 với các nhóm endpoint chính trong Bảng 6.

Table 6. Các endpoint chính của Core Service

Route	Method	Mục đích
/api/tx/deploy	POST	Tải lên contract WASM (multipart)
/api/tx/submit	POST	Gửi giao dịch: chạy → ký → sắp xếp
/api/contracts	GET	Liệt kê contract đã deploy
/api/blocks	GET	Liệt kê block đã commit
/api/transactions	GET	Liệt kê giao dịch đã commit
/api/explorer/stream	GET (SSE)	Đẩy block/giao dịch mới realtime
/api/metrics/benchmark	GET	Tổng hợp submit + commit + E2E latency
/api/metrics/throughput	GET	Tốc độ commit theo cửa sổ

Endpoint `/api/explorer/stream` dùng **Server-Sent Events (SSE)** — server đẩy dữ liệu một chiều xuống trình duyệt qua một kết nối HTTP giữ mở. Khi có block/giao dịch mới, Core gửi sự kiện `ledger_update`. SSE được chọn thay WebSocket vì luồng dữ liệu ở đây một chiều (server→client), SSE đơn giản hơn và tự kết nối lại.

3.4 Ordering Service — Đồng thuận Raft

Đây là phần học thuật trọng tâm của khóa luận: **một hiện thực Raft tự viết hoàn toàn** (không dùng thư viện), với biến thể bầu lãnh đạo theo độ ưu tiên.

3.4.1 Vai trò và lý do cần sắp xếp thứ tự

Trong hệ phân tán, nhiều giao dịch đến gần như đồng thời từ nhiều nguồn. Nếu mỗi node tự quyết thứ tự, các bản sao sổ cái sẽ khác nhau, dẫn tới mất nhất quán. Ordering Service đảm bảo **mọi node thấy cùng một thứ tự giao dịch duy nhất** — gọi là *total order* (thứ tự toàn phần). Đây là điều kiện tiên quyết để world state hội tụ giống nhau ở mọi nơi.

3.4.2 Raft của dự án khác Raft chuẩn ở đâu?

Dự án giữ tinh thần Raft [5] nhưng thay đổi cơ chế bầu cử cho *xác định và đơn giản hơn*. Bảng 7 tóm tắt các khác biệt.

Table 7. So sánh Raft chuẩn và Raft của dự án

Khía cạnh	Raft chuẩn	Dự án này
Bầu Leader	Timeout ngẫu nhiên , ai hết giờ trước thì ứng cử	Theo độ ưu tiên : node tham gia sớm hơn = ưu tiên cao hơn = được chọn xác định
Trạng thái	Follower / Candidate / Leader	Follower / ClaimingLeader / Leader / Syncing
Phiếu bầu	Mỗi node bầu cho một candidate	Follower ACK lời tuyên bố “Tôi là Leader mới”
Nội dung log	Lệnh tùy ý	Mỗi entry chứa một block
Lưu bền	Ghi đĩa (term, log)	Toàn bộ trong RAM (nhanh, nhưng mất khi crash)
Thư viện	etcd/raft, hashicorp/raft	Tự viết hoàn toàn

Vì sao bầu theo độ ưu tiên? Trong Raft chuẩn, nhiều node có thể cùng ứng cử và chia phiếu (split vote), buộc phải bầu lại, làm chậm hội tụ. Priority-based loại bỏ điều này vì luôn có một node “đáng làm Leader nhất” được xác định trước; ai cũng biết trước thứ tự ưu tiên nên ít bất định, dễ suy luận và debug. Đánh đổi là thiết kế này thiên về tính đơn giản và tốc độ hội tụ hơn là tổng quát hóa.

3.4.3 Các trạng thái của node

Mỗi node ở một trong bốn trạng thái, chuyển đổi như Hình 3.

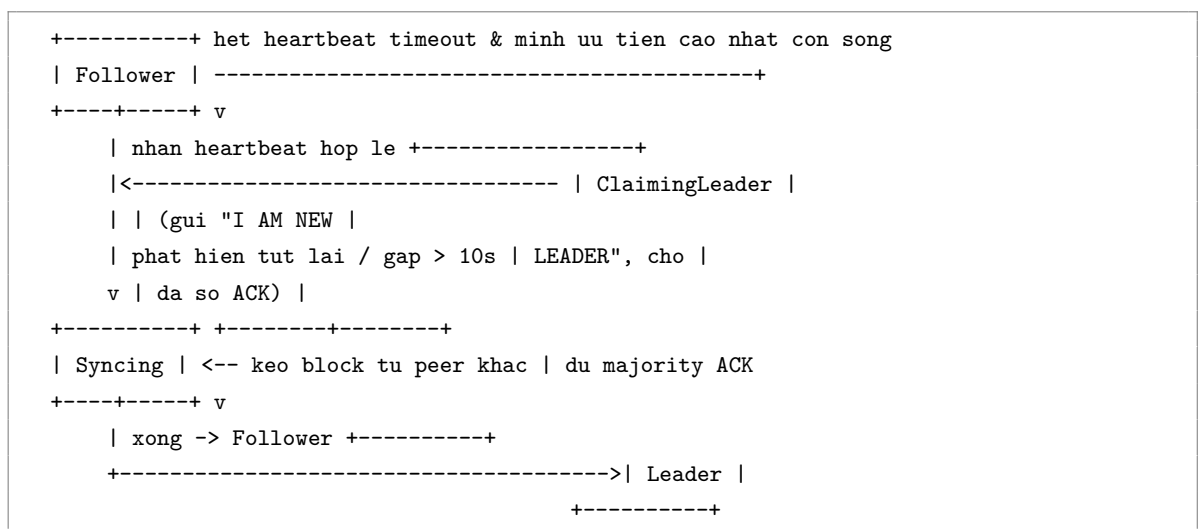


Fig. 3. Sơ đồ chuyển trạng thái của một node Raft trong dự án.

- **Follower:** trạng thái mặc định; nghe heartbeat, nhận và ACK block proposal.
- **ClaimingLeader:** đang tự tuyên bố làm Leader, chờ thu đủ ACK đa số.
- **Leader:** gom giao dịch, cắt block, điều phối commit, gửi heartbeat.
- **Syncing:** đang kéo block bị thiếu từ peer khác.

3.4.4 Quorum tính trên tổng số thành viên

Một quyết định an toàn then chốt: quorum đa số được tính trên **tổng số thành viên** (cả node sống lẫn chết), không chỉ trên số node đang sống:

```
totalCount := Membership.GetTotalCount() // ca song lan chet
majority := totalCount/2 + 1
```

Lý do là để **chống split-brain** (não phân ly): nếu mạng chia đôi 50–50, không bên nào đạt quá bán trên tổng, nên không thể có hai Leader cùng commit ở hai phía. Đây là nguyên tắc an toàn nền tảng của Raft, dựa trên tính chất hai tập đa số bất kỳ luôn giao nhau.

3.4.5 Bầu lãnh đạo và heartbeat

Độ ưu tiên = thứ tự tham gia Mỗi thành viên cluster có một Priority (số nguyên), gán theo thứ tự tham gia: tham gia càng sớm, số càng nhỏ, càng được ưu tiên. Hàm `GetHighestPriorityAliveNode()` trả về node sống có priority nhỏ nhất — chính là ứng viên Leader hợp lệ.

Table 8. Các hằng số thời gian của tầng đồng thuận

Hằng số	Giá trị	Ý nghĩa
HeartbeatInterval	2s	Leader gửi heartbeat nếu 2s chưa gửi block nào
HeartbeatTimeout	5s	Follower quá 5s không nghe Leader → bầu cử
DetectionTimeout	3s	Ngưỡng phát hiện (ticker chạy mỗi 1s)
Cửa sổ chờ ACK	10s	Thời gian thu ACK đa số khi tuyên bố Leader
Hạn Leader kỳ vọng	15s	Follower chờ node ưu tiên cao nhất ra tuyên bố

Các hằng số thời gian Lưu ý: block proposal/commit cũng đóng vai trò heartbeat — khi cluster đang bận cắt block thì không cần gửi heartbeat riêng (tiết kiệm mạng).

Quy trình bầu Leader Khi Follower phát hiện Leader mất (qua vòng giám sát `monitorHeartbeat()` chạy mỗi 1 giây), quy trình bầu cử diễn ra như Hình 4.

```
Follower phát hiện Leader chết
|
v
selectNewLeader() -- mình là ưu tiên cao nhất? --+ Không -> chờ node kia (<=15s)
| Co |
v v
ClaimingLeader: term++, broadcast "I AM NEW LEADER"
|
v
thư ACK -- đủ majority YES? --+ Không -> về Follower
| Co
v
trở thành Leader
```

Fig. 4. Quy trình bầu lãnh đạo theo độ ưu tiên.

Cụ thể: (1) `selectNewLeader()` đánh dấu Leader cũ chết, chọn node ưu tiên cao nhất còn sống; nếu chính mình là node đó thì chuyển sang `ClaimingLeader`, nếu không thì chờ node kia ra tuyên bố trong hạn 15s. (2) `sendIAmNewLeaderAndWaitForAck` tăng term, phát `MsgIAmNewLeader` tới mọi thành viên, chờ tối đa 10s thu các `MsgLeaderClaimAck`. (3) Follower trả YES/NO dựa trên ba điều kiện: người tuyên bố có đúng là node ưu tiên cao nhất còn sống không; term có $\geq$ `currentTerm` của mình không; và Leader hiện tại có thực sự đã “ôi” (stale) không. (4) `finishClaim()`: nếu thu được $\geq$ majority ACK YES thì trở thành Leader, ngược lại quay về Follower.

3.4.6 Nhân bản log và cắt block

Đây là “đường nóng” (hot path) quyết định throughput của hệ thống.

TxPool và vòng cắt block lai Leader nhận giao dịch và gom vào một **TxPool** (hồ chứa trong RAM). Vòng cắt block (**StartAutoProposeBlock()**) hoạt động theo cơ chế **lai (hybrid)** vừa theo sự kiện vừa theo timeout:

```
lap mai:
    neu len(TxPool) >= AutoProposeBlockSize (1000):
        -> cat block NGAY (khong cho)
    nguoc lai:
        -> cho toi da AutoProposeInterval (100ms) roi xa phan con lai
sau khi propose:
    -> cho block truoc commit xong (hoac timeout 10s) moi propose tiep
```

Listing 3.5. Vòng cắt block lai

Đánh đổi kích thước lô: block lớn → ít block hơn, throughput cao hơn (chi phí đồng thuận chia cho nhiều giao dịch); nhưng nếu tải thấp, phải chờ tới 100 ms mới cắt → latency tăng. Cơ chế lai cân bằng: tải cao thì cắt ngay theo kích thước, tải thấp thì xả theo thời gian.

Đề xuất, ACK và commit Khi cắt block, Leader: (1) lấy N giao dịch đầu từ TxPool, tính **Merkle root** từ các txid (với block > 1000 tx, việc tính Merkle được song song hóa), tính **Hash** = double-SHA256 của header, đặt **PrevHash** = **lastCommittedHash**; (2) tạo **LogEntry** với **Index**, **PrevLogIndex**, **Term**, append vào **RaftLog**; (3) phát **MsgBlockProposal** tới mọi **Follower**. **Follower** kiểm tra người gửi là **Leader**, **term** >= **currentTerm**, và *kiểm tra liên tục log* (**entry.PrevLogIndex** == **lastIndex** của mình); nếu khớp thì append và gửi **MsgBlockProposalAck**, nếu lệch thì từ chối (phải sync). Khi Leader đếm đủ majority ACK, nó **commit**: thêm block vào **OrderingBlock**, cập nhật **lastCommittedHash**, gọi **DeliverMgr.NotifyNewBlock()**, xóa giao dịch đã đóng block khỏi TxPool, và phát **MsgBlockCommit** cho **Follower**.

Các tối ưu tốc độ Một loạt tối ưu trên hot-path (theo docs/block-speed-optimization) đã được thực hiện, tóm tắt trong Bảng 9.

Table 9. Các tối ưu tốc độ trên đường nóng

Mã	Nội dung	Trạng thái
OPT-1	Commit ngay khi đủ majority ACK (không chờ ticker)	Đã làm
OPT-2	Propose theo sự kiện (đủ batch → cắt ngay)	Đã làm
OPT-3	Stream endorsement bền (mỗi worker giữ 1 stream, gửi nhiều tx)	Đã làm
OPT-5	Tách kênh riêng cho hot-path (ACK → BlockAckChan)	Đã làm
OPT-8	Bỏ log theo từng tx trên hot-path, chỉ log theo block	Đã làm

Kết quả quan sát: sau OPT-8, mỗi block có thể đạt 1000+ tx ở tải bền vững (trước đó nghẽn ~ 71 tx/block do tranh chấp khóa khi log từng tx). Các hạn chế còn lại gồm: vẫn **1 block in-flight** (propose → ACK → commit tuần tự, chưa pipeline) — giới hạn throughput ở mức $1/RTT-commit$; còn marshal/unmarshal lặp; và **không lưu bền** — toàn bộ log/block trong RAM, crash là mất.

3.4.7 Giao block (Deliver) và Đồng bộ (Sync)

Deliver — mô hình đăng ký & phát tỏa `DeliverManager` quản lý danh sách **subscriber** (mỗi `Committing Peer` là một subscriber). `subscribe()` tạo một channel có đệm (cap 64); `NotifyNewBlock(block)` gửi block tới mọi subscriber, và nếu subscriber chậm (channel đầy) thì block bị **bỏ qua không chặn** (non-blocking) để không làm nghẽn luồng commit — peer chậm sẽ tự bắt kịp bằng sync. Khi một `Committing Peer` kết nối qua protocol `/raft-order-service/deliver` nó gửi `DeliverRequest{FromIndex}`; orderer gửi ngay tất cả block đã commit từ `FromIndex`, rồi `subscribe()` để stream tiếp block tương lai. Kết nối giữ mở lâu dài — đây là *streaming push*, peer không phải hỏi liên tục.

Sync — **đồng bộ bốn pha** Khi một orderer mới tham gia hoặc vừa hồi sinh, nó có thể thiếu nhiều block. `sync.go` lo việc “đuổi kịp” theo mô hình **pull (kéo)**. Sync gồm bốn pha:

1. **Khám phá ($\sim 2s$):** phát `MsgSyncStatusRequest` kèm `commitIndex` của mình; thu `MsgSyncStatusResponse` từ các peer (gồm `commitIndex`, `commitHash`, `logLastIndex`, `leaderID`, phiên bản `membership`).
2. **Chọn nguồn:** nhóm các phản hồi theo cặp (`commitIndex`, `commitHash`); chọn nhóm có **nhiều phiếu nhất** (đồng thuận mạnh nhất), hòa thì ưu tiên

`commitIndex` cao hơn; ưu tiên Leader trong nhóm thắng làm nguồn tải. Việc bỏ phiếu theo (`commitIndex`, `commitHash`) đảm bảo node sync từ *trạng thái đa số đồng ý*, không bị một node lỗi đánh lừa.

3. **Tải block song song:** chia khoảng [`from..to`] thành các shard (mỗi shard 64 block); tải các shard song song từ nhiều nguồn (round-robin cân tải), thất bại thì thử nguồn kế.
4. **Xác minh & cài đặt:** `verifyHashChain()` kiểm tra mỗi block có `PrevHash` khớp Hash block trước, block cuối khớp `commitHash` mục tiêu; sai thì sync thất bại và thử lại. Nếu đúng thì append block, cài log entry, cập nhật `lastCommittedHash`, chuyển Syncing → Follower.

Về an toàn, node đang tự sync sẽ *từ chối phục vụ* sync cho node khác để không lan truyền dữ liệu cũ.

3.4.8 Networking

Tầng vận chuyển (`transport.go`) bọc một libp2p host với các thao tác `SendMessage`, `BroadcastMessage` (gửi bất đồng bộ kèm callback xử lý lỗi để đánh dấu peer chết) và `NewStream`. Bảng 10 liệt kê các Protocol ID dùng trong toàn hệ thống.

Table 10. Các Protocol ID libp2p dùng trong hệ thống

Protocol ID	Mục đích
/raft-order-service/1.0.0	Thông điệp consensus (heartbeat, bầu cử, membership)
/raft-order-service/endorsement/1.0.0	Core → Orderer: gửi tx đã endorse
/raft-order-service/deliver/1.0.0	Orderer → Committing Peer: stream block
/raft-order-service/sync/1.0.0	Đồng bộ block/log giữa các orderer
/fabric-enterprise/commit-peer/tx-sign/1.0.0	Core → Committing Peer: xin chữ ký
/committing-peer/sync/1.0.0	Client → Committing Peer: truy vấn UTXO

Hai tối ưu hot-path trên mạng đặc biệt quan trọng. **OPT-3 (stream endorsement bền):** thay vì mở một stream mới cho *mỗi* giao dịch (gây ~10.000 lần mở stream/giây ở 5000 TPS), `HandleEndorsementStream()` đọc nhiều tx trong một vòng lặp `for { decoder.Decode(&tx) }` trên **một stream giữ mở**; loadgen workers cũng giữ một stream/worker. Đây là khác biệt lớn nhất giúp

vượt nghìn cổ chai mở stream. **OPT-5 (bỏ qua kênh chung):** `MsgBlockProposalAck` đi thẳng vào `BlockAckChan` riêng, tránh dồn mọi message qua một hàng đợi chung gây tranh chấp.

3.4.9 Công cụ bắn tải (loadgen) và client

Bộ máy bắn tải Go (`pkg/loadgen/`) bơm giao dịch ở tốc độ cao (mặc định 5000 TPS) để đo hiệu năng, với tham số `-orderer <multiaddr> -tps 5000 -duration 30s -workers 16`. Nó tái dùng stream theo OPT-3 và theo dõi sự kiện `block-commit` để tính latency. CLI client (`cmd/client`) là một ví UTXO đầy đủ: sinh địa chỉ từ khóa Ed25519, theo dõi UTXO, tạo và ký giao dịch. Khác với loadgen (bắn thẳng tới orderer qua libp2p, đo throughput consensus thuần), công cụ k6 bắn HTTP tới Core Service (đo toàn pipeline submit và độ trễ HTTP từ góc nhìn client thật).

3.5 Committing Peer — Kiểm tra và Ghi sổ

3.5.1 Vai trò và kiến trúc pipeline

Committing Peer (`committingpeer/`, Go, metrics :8081) là trạm cuối của pipeline: nhận block đã sắp xếp, kiểm tra hợp lệ, rồi ghi vĩnh viễn. File `chain.block` và LevelDB ở đây mới là sổ cái thật. Quan trọng: Committing Peer **không** chạy thuật toán đồng thuận — nó *tin tưởng* rằng block từ Ordering Service đã là thứ tự cuối, việc của nó là kiểm tra tính hợp lệ mật mã rồi ghi.

Bên trong, ba tầng được tách rời bằng channel để mỗi tầng chạy với tốc độ riêng (Hình 5).

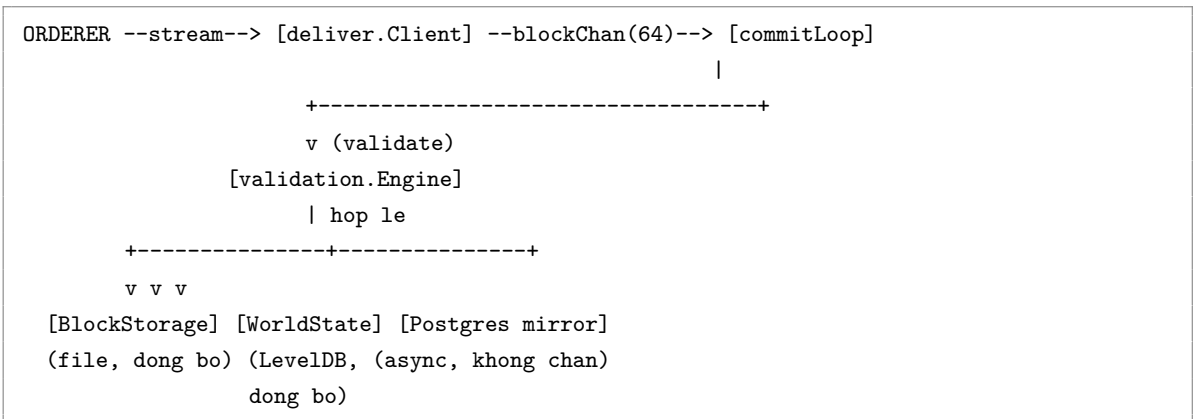


Fig. 5. Kiến trúc đường ống ba tầng của Committing Peer.

3.5.2 Nhận block và tự kết nối lại

Committing Peer dựng libp2p host, đăng ký nhận block qua `/raft-order-service/deliver`. Luồng `Subscribe()` parse multiaddr orderer, kết nối, mở stream, gửi `DeliverRequest{FromIndex, Block}` (“tôi đã có tới block X, gửi tiếp từ X+1”), rồi một goroutine nền lặp `Decode(&block)` và đẩy vào `blockChan`. Khi Leader đổi hoặc stream đứt, `deliverReconnectLoop()` khám phá orderer mới và kết nối lại với `fromIndex = lastCommittedBlock + 1` — **không mất block nào** — dùng exponential backoff (1s → tối đa 30s) để tránh dội kết nối.

Committing Peer cũng đóng vai **endorser**: đăng ký handler `/fabric-enterprise/comm` khi Core gửi một giao dịch chưa ký, peer ký Ed25519 trên `txid + contractName + payload`, thêm một `EndorsementEntry`, rồi trả lại. Khóa công khai của nó nằm trong danh sách endorser tin cậy (`TRUSTED_ENDORSER_PUBLIC_KEYS`).

3.5.3 Kiểm tra hợp lệ (Validation)

Trước khi ghi, `ValidateBlock()` kiểm tra hai phần. **Toàn vẹn cấu trúc** (`verifyBlockIntegrity()`): tính lại Merkle root từ các `txid` và so với `MerkleRoot` của block; tính lại block hash (double-SHA256 của header) và so với `Hash`; lệch thì từ chối. Việc tính lại độc lập đảm bảo block không bị sửa đổi trên đường truyền, và hàm hash/merkle phải khớp y hệt cách Ordering Service tính. **Kiểm tra endorsement** (`validateEndorsedTx()`): với mỗi giao dịch smart-contract, phải có mảng `Endorsements[]` không rỗng, xác minh từng chữ ký bằng `VerifyTransaction`, và ít nhất một khóa endorser phải nằm trong tập tin cậy.

Phạm vi hiện tại (theo mã nguồn): một số kiểm tra chưa được thực thi đầy đủ trên hot-path. Việc kiểm tra liên tục chuỗi (`PrevHash` của block khớp `Hash` block trước) hiện đang bị tắt ở engine — block hash vẫn *bao gồm* `prevHash` để toàn vẹn cấu trúc, nhưng liên kết chuỗi chỉ được xác minh đầy đủ ở luồng `sync` của `orderer`. Hàm `ValidateTransaction()` mức từng giao dịch hiện là *stub* (kiểm tra double-spend UTXO, chữ ký input chưa thực thi đầy đủ). Đây là các điểm rủi ro được ghi nhận để cải thiện về bảo mật. Về tính kiên cường, một block không hợp lệ chỉ bị ghi log mà không làm sập pipeline — peer tiếp tục xử lý block sau.

3.5.4 Lưu trữ: ba lớp

Committing Peer giữ ba lớp lưu trữ, mỗi lớp một mục đích (Bảng 11).

Table 11. Ba lớp lưu trữ của Committing Peer

Lớp	Công nghệ	Đặc tính	Mục đích
Block storage	File append-only	Bất biến, ghi tuần tự	Sổ cái thật, kiểm toán
World state	LevelDB	Ghi đè, atomic batch	Tra UTXO hiện tại
Mirror	PostgreSQL	Index, JSONB, async	Explorer & báo cáo

Block storage: chuỗi block lưu trong một file append-only (`chain.block`), mỗi block là một dòng JSON. File mở với cờ `O_APPEND | O_CREATE | O_WRONLY`, **không bao giờ** truncate — đảm bảo “bất biến ở mức file”. Đọc lại dùng `bufio.Scanner` với buffer tới 16 MB (chứa được block nhiều giao dịch).

World state: UTXO set lưu trong LevelDB. Hàm `ApplyBlock()` cập nhật **nguyên tử**: với mỗi block tạo một `leveldb.Batch`, mỗi VIN là `batch.Delete`, mỗi VOUT là `batch.Put`, rồi `db.Write(batch)` — một thao tác tất-cả-hoặc-không. Điều này quan trọng vì nếu ghi nửa chừng (xóa input nhưng chưa tạo output) rồi crash, world state sẽ sai.

PostgreSQL mirror: bản sao bất đồng bộ, không nằm trong luồng commit. Block được coi là *đã commit ngay khi ghi xong file + world state*; việc ghi PostgreSQL diễn ra sau, ở các worker nền (hàng đợi mặc định 512, số worker

mặc định 2). Nếu hàng đợi đầy thì sinh goroutine ghi ngay (non-blocking). **Triết lý:** tốc độ commit blockchain không bị phụ thuộc vào tốc độ ghi PostgreSQL.

3.5.5 Metrics — nguồn ground truth

Vì Committing Peer là nơi giao dịch được commit thật, nó là nguồn “sự thật mặt đất” tốt nhất để đo throughput và E2E latency. `recorder.go` ghi thời điểm commit *ngay trong RAM* (map `txid` → `thời điểm`), không lệ thuộc độ trễ ghi PostgreSQL nên chính xác hơn khi benchmark. Server metrics tại `:8081` cung cấp `/metrics/throughput`, `/metrics/benchmark` và `/metrics/commit-lookup`; Core Service gọi các endpoint này để tổng hợp benchmark E2E.

3.6 Blockchain Explorer — Giao diện

3.6.1 Tech stack và kiến trúc

Blockchain Explorer (`BlockchainExplorer-FrontEnd/`) là một ứng dụng web một trang (SPA) viết bằng React, có hai chức năng: khám phá (xem block và giao dịch đã commit theo thời gian thực) và gửi giao dịch mới. Nó nói chuyện với Core Service (HTTP `:8080`) qua proxy của Vite. Bảng 12 liệt kê các công nghệ chính.

Table 12. Công nghệ của Blockchain Explorer

Thư viện	Phiên bản	Vai trò
React	18.3.1	Framework UI (hooks)
React Router DOM	6.26.2	Điều hướng client-side
Vite	5.4.1	Build tool & dev server (proxy <code>/api</code>)
Material-UI (MUI)	6.1.1	Bộ component giao diện
Tailwind CSS	3.4.13	CSS tiện ích
CryptoJS	4.1.1	Băm SHA-256 (tạo txid phía client)

Ứng dụng dùng React Router với ba tuyến đường (`/transactions`, `/transfer`, `/blocks`), tất cả render qua Dashboard với prop `section` khác nhau. Lớp gọi API (`src/api/client.js`) dùng URL tương đối được Vite proxy chuyển tới Core Service, tránh vướng CORS khi dev.

3.6.2 Mã hóa payload nhị phân

Payload của giao dịch cần nhỏ gọn và đúng kiểu để hợp đồng đọc. Frontend có hai cơ chế mã hóa, đều xuất ra chuỗi hex: (1) **hệ nhị phân tùy biến** (`transactionTypes.js`) đóng gói từng trường thành byte (big-endian, với các kiểu `STRING`, `UINT8/16/32/64`, `FLOAT/DOUBLE`, `BYTES`, `BOOLEAN`, `ADDRESS`), tiết kiệm $\sim 70\%$ so với JSON; và (2) **hệ payload contract** (`transactionUtils.js`) JSON $\rightarrow$ UTF-8 $\rightarrow$ hex, để contract Go dùng `json.Unmarshal` đọc. Vì contract WASM hiện đọc payload bằng `json.Unmarshal`, form Transfer dùng cơ chế JSON-hex khi gửi tới contract. Hàm `coercePayloadFieldsBySchema` ép kiểu input HTML (luôn là chuỗi) về int/float đúng theo schema trước khi mã hóa.

3.6.3 Cập nhật realtime qua SSE

Explorer dùng SSE để hiện block/giao dịch mới ngay khi commit. Khi Dashboard mount, nó mở `EventSource('/api/explorer/stream')`; nhận sự kiện `ledger_update` (chứa `latest_block` và `latest_tx`) thì cập nhật (upsert) danh sách. Nếu SSE đứt, Dashboard chuyển sang **polling dự phòng** (gọi `/api/transactions` và `/api/blocks` mỗi 3000ms), và dừng polling khi SSE kết nối lại — mẫu “ưu tiên đẩy, có lưới đỡ kéo”. Trạng thái kết nối được hiển thị bằng chấm màu (xanh/vàng/đỏ).

3.7 Cơ sở dữ liệu PostgreSQL

Cần nhấn mạnh lại: PostgreSQL **không** nằm trong luồng đồng thuận; số cái thật là `chain.block` + LevelDB ở Committing Peer. PostgreSQL là bản sao phục vụ hiển thị Explorer, lưu danh mục contract và đo benchmark. Dữ liệu được tách thành **ba schema** theo đúng ranh giới trách nhiệm của ba dịch vụ, như Bảng 13.

Table 13. Ba schema của cơ sở dữ liệu

Schema	Ai ghi	Nội dung
<code>core_service</code>	Core Service	Contract, thời điểm submit
<code>order_service</code>	Orderer (tùy chọn)	Giao dịch & block đã sắp xếp
<code>commit_peer</code>	Committing Peer	Số cái đã commit (mirror)

Các bảng chính: `core_service.smart_contracts` (bytecode WASM + payload_schema JSONB); `core_service.tx_submit_times` (txid PRIMARY KEY, submitted_at, index theo submitted_at DESC để đo E2E); `commit_peer.ledger` (block_hash UNIQUE, block_number, block_data JSONB, committed_at); và `commit_peer.ledger` (liên kết block_id → txid, tx_data JSONB, khóa ngoại ON DELETE CASCADE). Lưu ý UTXO world state **không** ở PostgreSQL — nó nằm trên đĩa (LevelDB) tại Committing Peer. Các thao tác chèn dùng ON CONFLICT DO NOTHING để idempotent.

3.8 Phương pháp đo lường hiệu năng

3.8.1 Ba lớp đo lường

Một giao dịch đi qua nhiều chặng; đo ở chặng khác nhau cho con số khác nhau. Khóa luận phân biệt rõ ba lớp đo (Bảng 14).

Table 14. Ba lớp đo lường hiệu năng

Lớp	Nguồn	Đo cái gì
k6 (client)	Script k6/submit-tx.js	Độ trễ HTTP, Core có trả 200 không
Submit (Core)	core_service.tx_submit_times	Lúc Core <i>chấp nhận</i> giao dịch
Commit (ledger)	commit_peer.ledger*	Lúc giao dịch <i>đã ghi sổ</i>

E2E latency = committed_at − submitted_at (ghép theo txid). Cần lưu ý submit_latency_ms của k6 (vài chục ms) **khác** latency_ms_p95 E2E (có thể vài giây khi có backlog): một cái đo “Core nhận”, một cái đo “cả pipeline”.

3.8.2 Hai chỉ số cốt lõi

Throughput (TPS): số giao dịch/giây, phân biệt *sustained* (bền vững — trung bình cả cửa sổ) và *peak* (đỉnh — nhiều nhất trong một giây). **Latency (độ trễ)**: đo bằng phân vị p50/p95/p99 (p95 nghĩa là 95% giao dịch nhanh hơn giá trị này). Hai chỉ số này thường đánh đổi nhau.

3.8.3 Hiện tượng backlog và định luật Little

Khi **tốc độ gửi** > **tốc độ commit bền vững**, giao dịch dồn vào TxPool và backlog tích lũy, làm E2E latency tăng dù Core chấp nhận nhanh — biểu hiện kinh điển của hệ thống đạt **điểm bão hòa** (saturation). Hiện tượng này được lý giải bằng **định luật Little** [9]: số việc tồn đọng trung bình $L = \lambda W$, với λ là tốc độ đến và W là thời gian lưu trú — khi λ vượt năng lực xử lý, W (và do đó độ trễ) tăng không giới hạn.

3.8.4 Kết quả thực nghiệm tham khảo

Trong phép đo tham khảo ($5000 \text{ req/s} \times 60\text{s}$), hệ thống đạt các con số trong Bảng 15.

Table 15. Kết quả benchmark tham khảo (5000 req/s, 60s)

Chỉ số	Giá trị
Submit sustained	$\sim 4.691/\text{s}$ (đỉnh $\sim 5.017/\text{s}$)
Commit sustained	$\sim 3.921/\text{s}$ (đỉnh $\sim 6.000/\text{s}$)
Trung bình tx/block	~ 989 (batch ~ 1.000)
E2E p95	$\sim 7,4\text{s}$ (khi gửi > commit $\rightarrow$ backlog)

Diễn giải: Core nhận gần đủ 5000/s, nhưng commit “trần” $\sim 3.900/\text{s}$ bền vững, nên chênh lệch $\sim 800/\text{s}$ tạo backlog và đẩy p95 lên mức giây. Nguyên nhân gốc là giới hạn “1 block in-flight” (Mục 3.4.6): chu trình propose $\rightarrow$ ACK $\rightarrow$ commit còn tuần tự, nên throughput bị chặn ở mức $1/RTT\text{-commit}$. Đây chính là động lực cho hướng cải thiện quan trọng nhất: **pipeline nhiều block in-flight** — sẽ được phân tích ở chương đề xuất cải thiện.

3.9 Tổng kết công nghệ

Bảng 16 tổng kết ánh xạ giữa các vấn đề kỹ thuật và công nghệ được chọn để giải quyết, khép lại bức tranh thiết kế tổng thể.

Table 16. Ánh xạ “vấn đề ↔ công nghệ”

Vấn đề	Công nghệ
Các node nói chuyện an toàn	libp2p + Ed25519
Đồng ý cùng một thứ tự giao dịch	Raft (tự cài)
Chạy logic nghiệp vụ an toàn, đa ngôn ngữ	WASM (wazero + TinyGo)
Lưu trạng thái nhanh, nhúng sẵn	LevelDB
Dữ liệu không thể sửa	File append-only + hash chain
Tra cứu & báo cáo linh hoạt	PostgreSQL
Hiển thị realtime cho người dùng	React + SSE
Đo hiệu năng dưới tải	k6 + loadgen

Toàn bộ hệ thống được hiện thực bằng Go (ba dịch vụ backend) và React (giao diện), giao tiếp qua libp2p, với Raft tự cài đặt làm trái tim đồng thuận. Thiết kế tách tầng Execute–Order–Validate, kết hợp các tối ưu hot-path và cơ chế mirror bất đồng bộ, cho phép hệ thống tiệm cận mục tiêu 5.000 TPS trong khi vẫn giữ được tính module hóa và khả năng chịu lỗi — đồng thời bộc lộ rõ các điểm đánh đổi và hạn chế làm nền cho các đề xuất cải thiện ở những chương tiếp theo.

References

1. S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” <https://bitcoin.org/bitcoin.pdf>, 2008, whitepaper.
2. Bộ Chính trị, “Nghị quyết số 57-nq/tw về phát triển khoa học, công nghệ, đổi mới sáng tạo và chuyển đổi số quốc gia,” 2024, ngày 22/12/2024.
3. Thủ tướng Chính phủ, “Quyết định số 1236/qĐ-ttg về chiến lược quốc gia ứng dụng và phát triển công nghệ blockchain đến năm 2030,” 2023, ngày 30/6/2023.
4. E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich *et al.*, “Hyperledger fabric: A distributed operating system for permissioned blockchains,” in *Proceedings of the Thirteenth EuroSys Conference (EuroSys '18)*. ACM, 2018, pp. 1–15.
5. D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC '14)*, 2014, pp. 305–319.
6. V. Buterin, “A next-generation smart contract and decentralized application platform,” <https://ethereum.org/en/whitepaper/>, 2014, ethereum Whitepaper.
7. M. Castro and B. Liskov, “Practical byzantine fault tolerance,” in *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI '99)*, 1999, pp. 173–186.
8. M. J. Fischer, N. A. Lynch, and M. S. Paterson, “Impossibility of distributed consensus with one faulty process,” *Journal of the ACM*, vol. 32, no. 2, pp. 374–382, 1985.
9. J. D. C. Little, “A proof for the queuing formula: $l = \lambda w$,” *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.
10. National Institute of Standards and Technology, “Secure hash standard (shs),” FIPS PUB 180-4, 2015.
11. R. C. Merkle, “Secrecy, authentication, and public key systems,” Stanford University, Tech. Rep., 1979, ph.D. dissertation.
12. G. Wood, “Ethereum: A secure decentralised generalised transaction ledger,” *Ethereum Project Yellow Paper*, vol. 151, pp. 1–32, 2014.

13. L. Lamport, “The part-time parliament,” *ACM Transactions on Computer Systems*, vol. 16, no. 2, pp. 133–169, 1998.
14. L. Lamport, R. Shostak, and M. Pease, “The byzantine generals problem,” *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, 1982.
15. C. Cachin and M. Vukolić, “Blockchain consensus protocols in the wild,” *arXiv preprint arXiv:1707.01873*, 2017.
16. A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. Bastien, “Bringing the web up to speed with webassembly,” in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI ’17)*, 2017, pp. 185–200.
17. W3C WebAssembly Working Group, “Webassembly core specification,” <https://www.w3.org/TR/wasm-core-1/>, 2019.
18. Bytecode Alliance, “Wasi: The webassembly system interface,” <https://wasi.dev/>, 2023.
19. Protocol Labs, “libp2p: A modular network stack,” <https://libp2p.io/>, 2023.
20. J. Benet, “Ipfps - content addressed, versioned, p2p file system,” *arXiv preprint arXiv:1407.3561*, 2014.
21. J. Dean and S. Ghemawat, “Leveldb: A fast key-value storage library,” <https://github.com/google/leveldb>, 2011.
22. T. T. A. Dinh, J. Wang, G. Chen, R. Liu, B. C. Ooi, and K.-L. Tan, “Block-bench: A framework for analyzing private blockchains,” in *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD ’17)*, 2017, pp. 1085–1100.
23. D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, “High-speed high-security signatures,” *Journal of Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, 2012.
24. S. Josefsson and I. Liusvaara, “Edwards-curve digital signature algorithm (eddsa),” <https://datatracker.ietf.org/doc/html/rfc8032>, 2017, rFC 8032.